



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

School of Engineering

Design, Implementation, and Validation of a Networked Control System for a Micro-Integrated Coherent Optical Receiver

Ruairi Mullally

Supervisor: Dr. Aleksandra Kaszubowska

April 12, 2026

A final year project submitted in partial fulfilment
of the requirements for the degree of
BAI (Computer and Electronic Engineering)

Declaration

I hereby declare that this final year project is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at <http://tcd-ie.libguides.com/plagiarism/ready-steady-write>.

I consent to the examiner retaining a copy of the thesis beyond the examining period, should they so wish (EU GDPR May 2018).

I agree that this thesis will not be publicly available, but will be available to TCD staff and students in the University's open access institutional repository on the Trinity domain only, subject to Irish Copyright Legislation and Trinity College Library conditions of use and acknowledgement.

The use made of AI tools in the work described here, including the preparation of the document itself, is outlined in Appendix A1.4.

Signed: *Ruairi Mullally*

Date: 12/04/2026

Abstract

A single misstep in a power-up sequence: applying 3.3 V to an amplifier before photodiodes are biased can permanently damage a Fujitsu FIM24725 micro-integrated coherent optical receiver. In the Open Ireland CONNECT laboratory in Trinity, this sequencing constraint was previously enforced by a written procedure and operator memory. The receiver's 34-pin evaluation board needed to be hand-wired to power supplies, with each session requiring the operator to identify the correct channels, set protection limits and bring 7 voltage rails up in the correct order. This process was slow, error-prone, and inaccessible to anyone not physically in the laboratory.

This project replaces that manual workflow with a networked control system that makes the unsafe sequence structurally impossible. The system is built as a 4-layer software architecture providing synchronous, transport-isolated control of two Multicomp Pro MP710508 power supplies, that maps the receiver's 34 physical pins to seven named rails behind a five-state finite state machine, enforcing sequencing, validating every setpoint against manufacturer limits, and verifying every hardware command. A REST API exposes the service, which is driven by a Dash web interface providing real-time telemetry, state-driven controls, and operational logging. The complete system is containerised with Docker and deployed on a co-located Raspberry Pi 3, accessible remotely over Tailscale VPN. Total hardware cost was approximately €143, and all software used was open-source, all of which amounts to substantial cost and complexity savings over alternatives discussed.

The system was validated through fault injection testing on real hardware. A coherent detection experiment confirmed correct receiver operation under system control: a tuneable laser source was mixed with a local oscillator across wavelength offsets, producing well-defined electrical beat frequencies at 1.6–4.9 GHz that tracked the theoretical prediction with a systematic offset of 0.19–0.34 GHz likely attributable to laser drift and other environmental factors. The system maintained stable operation across multiple sessions, and a laboratory colleague was added to the Tailscale network and was operating the receiver independently within minutes.

The result is a plug-and-play box: power in, Ethernet in, fibres in, SMA out, that turns a fragile, expert-dependent experimental setup into a portable, remotely accessible laboratory instrument.

Lay Abstract

Fibre-optic networks carry nearly all of the world's internet traffic as pulses of light. At the receiving end, a device called a coherent optical receiver converts that light back into electrical signals that can be processed by computers. These receivers are sophisticated and expensive. The one used in this project, housed in the photonics laboratory at Trinity College Dublin, can be permanently destroyed if its power supplies are turned on in the incorrect order.

Previously, this procedure meant connecting dozens of wires to the correct sockets, and carefully following a step-by-step process to power on the receiver. Skipping a step, misplacing connections, inputting a wrong value, could destroy the device. The process took non-negligible time, specialist knowledge, and could only be done by somebody physically standing at the laboratory bench.

This project built a computer-controlled system to automate the process, and makes it impossible to damage the receiver by turning on power in the wrong order or setting a control incorrectly. The system is housed in a single box alongside the receiver, and is operated by a website accessible from a laptop, from any internet connection. A researcher opens a browser, clicks a start button, and the system handles the rest: programming safety limits, turning on power in the correct sequence, checking if hardware correctly responded at each step, and help shutting down if anything went wrong.

The system was tested by deliberately inducing faults, and running an experiment with the system, producing the expected results when two lasers of slightly different wavelengths were combined inside it. A colleague who had never used the system before was able to connect remotely and operate the receiver within minutes.

The entire system was built from inexpensive, off-the-shelf components and free open-source software, at a fraction of what commercial laboratory control solutions would require.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Aleksandra Kaszubowska, for her guidance, support, and shaping the requirements of this project. Her direction was invaluable throughout the development of this work.

I am especially grateful to Inés Cáceres Pablo for her extensive support and mentorship, particularly in the optical aspects of the project. Her expertise and hands-on assistance during the experimental phase were essential to the success of this work.

I would also like to thank Tajammul Ahmad Syed for his advice on optical equipment, as well as his help in setting up the experimental apparatus and addressing technical queries.

Additionally, I appreciate the guidance of Dmitrii Briantcev for his insights into software architecture, networking, and deployment, which significantly contributed to the implementation of this project.

I would like to acknowledge the CONNECT Centre Ireland for providing access to the Open Ireland CONNECT laboratory facilities and equipment in Trinity, which were essential for conducting the experimental work in this project.

Contents

1	Introduction	1
2	Background	3
2.1	Coherent Receiver Technology	3
2.1.1	Why do we need Coherent Detection? (Direct Detection versus Coherent Detection)	3
2.1.2	The Principles of Coherent Detection	4
2.1.3	The Fujitsu FIM24725 Micro-Integrated Coherent Receiver	5
2.2	Laboratory Instrument Control	6
2.2.1	SCPI and Command-Based Instrument Controls	6
2.2.2	Large-Scale Control Frameworks: EPICS	6
2.2.3	Middleground: GUI-Based Frameworks like LabVIEW	7
2.2.4	Other Modern Lightweight Approaches	7
2.2.5	The Gap	8
2.3	Software Concepts	8
2.3.1	Finite State Machines	8
2.3.2	Hardware Abstraction Layers	8
2.3.3	Separation of Concerns and Layered Architecture	8
2.3.4	REST APIs and Web-Based Interfaces	9
2.3.5	Containerisation	9
3	Design and Implementation	10
3.1	Design Requirements	10
3.1.1	Characterising the FIM24725	10
3.1.2	Safety-Critical Sequencing Constraints: Start-up & Shutdown Algorithms	12
3.1.3	Functional Requirements	14
3.1.4	Non-Functional Requirements	15
3.2	Hardware Platform	16
3.2.1	Power Supply Units (PSUs): Multicomp Pro MP710508	16
3.2.2	Microcontroller (MCU): Arduino Uno R3	18
3.2.3	Service Deployment Target: Raspberry Pi 3 Model B	19
3.2.4	System Topology	20
3.3	Software Design and Implementation	22
3.3.1	Hardware Abstraction Layer (HAL)	22
3.3.2	Service Layer: FIM24725Service	25
3.3.3	API Layer	33
3.3.4	Graphical User Interface (GUI)	36
3.3.5	Remote Access and Security	40
3.3.6	Deployment and Configuration	42
3.4	Taking a Step Back: Overall System Architecture	45

4	Evaluation	52
4.1	Coherent Detection Experiment	52
4.2	User Testimonial	54
4.3	Integration Testing	55
4.4	Comparison With Other Systems	55
4.5	Limitations	57
5	Conclusion	58
5.1	Summary of Achievements	58
5.2	Future Work	59
5.2.1	Wider Societal Implications	59
A1	Appendix	62
A1.1	MP710508 SCPI Schema	62
A1.2	Fujitsu Micro Integrated Coherent Receiver Application Note: Pin functions and power sequencing	63
A1.3	Requirements Traceability	66
A1.4	Use of GenAI in this work.	67

List of Figures

2.1	Internal structure of FIM24725 [1]	5
3.1	Annotated layout of the FIM24725 showing key components and interfaces [1].	10
3.2	Multicomp Pro MP710508 (image reproduced from [2])	16
3.3	OSEPP Arduino Uno R3 (image reproduced from [3])	18
3.4	Raspberry PI 3 Model B (image reproduced from [4])	20
3.5	Topology of Connected Physical System.	21
3.6	PSU HAL UML Class Diagram	23
3.7	Service Layer UML Class Diagram	26
3.8	Control System State Machine Diagram	27
3.9	FIM24725 Coherent Receiver API Endpoints	34
3.10	GUI — Landing page, system off.	37
3.11	GUI — Startup confirmation with initial mode selection.	38
3.12	GUI — READY state MGC mode, with control values set and readback visible.	38
3.13	GUI — FAULT during startup sequence.	39
3.14	GUI — Shutdown confirmation.	39
3.15	Tailscale Connection Paths (Direct and DERP Relay).	40
3.16	Network Access Zones for the Control System (Raspberry Pi).	41
3.17	Comprehensive System Architecture Diagram.	46
3.18	Example Data Flow Trace: Set VOA to 2.4 V.	47
3.19	System Layer Architecture Diagram	48
3.20	Custom Routing PCB Design Gerber.	49
3.21	Custom Routing PCB 3D View.	50
3.22	Assembled enclosure showing external connections.	50
4.1	Coherent Detection Experimental Setup.	52
4.2	Normalised Optical Spectra of Lasers used in Experiment.	53
4.3	ESA Traces of Coherent Receiver Output from Experiment.	53

List of Tables

3.1	FIM24725 DC pin functions and operating ranges.	11
3.2	FIM24725 RF signal output pins.	12
3.3	FR1 — Power sequencing and lifecycle management.	14
3.4	FR2 — Input control.	14
3.5	FR3 — Telemetry and monitoring.	14
3.6	FR4 — User interface.	15
3.7	NFR1 — Safety, reliability, concurrency.	15
3.8	NFR2 — Maintainability, Usability and extensibility.	15
3.9	NFR3 — Security and Access.	16
3.10	PSU 1 Control Rail Mapping: Amplifier supply, diode bias supply, VOA.	17
3.11	PSU 2 Control Rail Mapping: Gain adjust and offset adjust.	17
3.12	MCU Control and Read Mapping: Digital control and analogue monitoring.	19
3.13	MCU serial protocol.	24
3.14	Service-layer exception hierarchy.	30
3.15	Service layer exception to HTTP status code mapping.	35
3.16	API rate limits per endpoint group.	36
3.17	System deployment configuration parameters.	43
3.18	Log files, contents, and rotation policy.	43
4.1	Experimental measured and theoretical frequency differences.	54
4.2	Control system bill of materials.	56
4.3	Comparison of control system approaches for a single-instrument laboratory deployment.	56

List of Algorithms

1	FIM24725 Startup Sequence	29
2	FIM24725 Shutdown Sequence	30

Nomenclature

Symbol	Description	Unit
c	Speed of light in vacuum	m/s
λ	Wavelength	nm
$\Delta\lambda$	Wavelength offset	nm
Δf	Beat frequency	GHz
R	Responsivity	A/W
V_{cc}	Amplifier supply voltage	V
V_{PD}	Photodiode bias voltage	V
$X_I(t)$	In-phase component, X-polarisation	V
$X_Q(t)$	Quadrature component, X-polarisation	V
$Y_I(t)$	In-phase component, Y-polarisation	V
$Y_Q(t)$	Quadrature component, Y-polarisation	V

Abbr.	Full Term	Abbr.	Full Term
ACL	Access Control List	LO	Local Oscillator
ADC	Analogue-to-Digital Converter	MCU	Microcontroller Unit
AGC	Automatic Gain Control	MGC	Manual Gain Control
API	Application Programming Interface	MPD	Monitor Photodiode
CORS	Cross-Origin Resource Sharing	OA	Output Amplitude (adjust)
DAC	Digital-to-Analogue Converter	OCP	Over-Current Protection
DSP	Digital Signal Processing	OVP	Over-Voltage Protection
EPICS	Experimental Physics and Industrial Control System	PBS	Polarisation Beam Splitter
ESA	Electronic Spectrum Analyser	PCB	Printed Circuit Board
ESD	Electrostatic Discharge	PI	Peak Indicator
FSM	Finite State Machine	PSU	Power Supply Unit
GA	Gain Adjust	REST	Representational State Transfer
GPPO	General Purpose Push-On	RF	Radio Frequency
HAL	Hardware Abstraction Layer	SCPI	Standard Commands for Programmable Instruments
HTTP	Hypertext Transfer Protocol	SD	Shutdown (FIM24725 pin)
ICR	Integrated Coherent Receiver	SMA	Sub-Miniature version A
IF	Intermediate Frequency	TIA	Transimpedance Amplifier
I/Q	In-phase / Quadrature	UDP	User Datagram Protocol
		USB	Universal Serial Bus
		VOA	Variable Optical Attenuator
		VPN	Virtual Private Network

1 Introduction

Coherent optical receivers are critical components in modern telecommunications infrastructure and photonics research. The Open Ireland CONNECT laboratory environment at Trinity includes a high-value coherent optical receiver, the Fujitsu FIM24725, used for coherent demodulation of optical signals and related photonics experiments. These integrated receivers require careful configuration and handling to operate safely. In its prior lab workflow, the set-up and operation of the receiver are cumbersome and operator-dependent, requiring manual set-up steps and repeated verification of configuration. It required detailed knowledge of the receiver's pinout and the power supply channel assignments. This raises two key problems:

1. **Risk of expensive errors:** incorrect rail values, invalid parameter combinations, and/or improper startup/shutdown sequencing can cause immediate circuit damage.
2. **Low reproducibility and no remote operation:** manual procedures are difficult to reproduce consistently across users and sessions. Researchers at partner institutions cannot benefit from shared resources, and experiments cannot be remotely monitored or adjusted.

In shared laboratory environments, these aforementioned issues translate into a broader engineering challenge: how to ensure safe, correct, and repeatable operation of expensive and fragile hardware when access is distributed across multiple users with varying levels of expertise. Large-scale scientific facilities have long addressed similar challenges through comprehensive control frameworks. Systems such as the Experimental Physics and Industrial Control System (EPICS). However, EPICS-style solutions are typically heavyweight, requiring significant configuration effort and infrastructure, and are often ill-suited to small-scale laboratory setups or single-instrument deployments.

At the opposite end of the spectrum, many laboratories employ ad-hoc scripting approaches based on instrument command standards such as SCPI, and more recent laboratory automation frameworks, such as LabVIEW-based instrument control place greater emphasis on data acquisition and experiment orchestration than on enforcing safety-critical operational procedures.

This project is motivated by the gap between these existing approaches. Specifically, it attempts to design a lightweight, software-centric control architecture that can provide a meaningful improvement in safety, robustness, and repeatability for a single high-risk instrument without incurring the complexity of large-scale control frameworks.

The initial project plan defined the following objectives:

1. Define safe operating constraints and procedures for the FIM24725 coherent receiver setup based on manufacturer guidance and lab practice.
2. Design a layered control architecture consisting of a web-based UI for technicians, an API-driven backend with formal parameter validation, and a hardware abstraction layer supporting real instruments and future simulation integration.
3. Implement a containerised control application, deployable on the lab server and accessible over an already in-use VPN (Tailscale).

4. Integrate ethernet-based control of the MP710508
5. Validate the system with controlled real hardware tests.

A stretch goal was identified: the implementation of basic characterisation assistance features such as scripted parameter sweeps with structured data logging. Logging features were implemented, and parameter sweeps were implemented for gain adjustment at the service level, however they were not implemented further up the stack and are therefore omitted.

All five primary objectives were met, with two deviations from the original plan. First, the deployment target changed from the laboratory's shared servers to a dedicated Raspberry Pi, as the server administrator was unavailable during the project period. This deviation proved beneficial, resulting in a self-contained portable unit. Second, an Arduino microcontroller was introduced to provide the digital control signals and analogue telemetry that the power supplies could not deliver, a requirement that was anticipated but not detailed in the project plan and was introduced as some of the implementation details were emerging.

This project delivers three concrete outcomes:

1. A working, deployed control system for the FIM24725 coherent receiver that automates safe power sequencing, enforces operating limits, and provides remote browser-based access packaged as a self-contained laboratory instrument.
2. Experimental verification of coherent detection under system control, demonstrating that the system enables stable and repeatable receiver operation across multiple sessions.
3. A reusable software architecture: layered, containerised, and platform agnostic, that provides a pattern for lightweight safety-critical instrument control applicable to other shared laboratory equipment.

Chapter 2 provides background on coherent optical detection, the FIM24725 receiver, laboratory instrument control paradigms, and the software concepts underpinning the control system design. Chapter 3 presents the design requirements from the device characterisation and laboratory context and describes the hardware platform and software architecture. Chapter 4 evaluates the system through fault injection testing, a coherent detection experiment, cost, and comparison with alternative approaches. Chapter 5 summarises the project outcomes and identifies directions for future work.

2 Background

2.1 Coherent Receiver Technology

2.1.1 Why do we need Coherent Detection? (Direct Detection versus Coherent Detection)

There are three fundamental stages to an optical communication system; a transmitter that encodes (modulates) data onto optical carriers; an optical fibre medium for transmission; a receiver that recovers the transmitted information. The most impactful feature for system performance is the type of encoding schema used, and the receiver architecture determines which schemas are available.

There are two principal detection types: direct and coherent. Direct detection measures only the instantaneous power (intensity) of the signal, operating as a square law detector responding to $|E(t)|^2$, where $E(t)$ represents the optical field [5, 6]. This inherently discards all phase information, which limits the receiver to intensity-modulated encoding formats such as on-off keying (OOK) or pulse amplitude modulation (PAM).

In contrast, coherent detection preserves both amplitude and phase components of the optical field through interference between the received signal and a local oscillator (LO) laser. The preservation of phase information enables advanced modulation formats including M-ary phase-shift keying (M-PSK) and quadrature amplitude modulation (QAM), which encode information in both amplitude and phase dimensions. These formats achieve significantly higher spectral efficiency, transmitting more bits per unit of optical bandwidth by utilizing multiple constellation points in the complex signal space [6]. The optical gain provided by mixing the weak signal with a strong LO significantly improves the sensitivity of coherent detectors compared to direct detectors, which extends the transmission reach over fibre, or allows transmission at lower powers [6]. Coherent detection also enables compensation of transmission impairments in the electrical domain through digital signal processing (DSP) for issues such as chromatic dispersion [6, 7]. The coherent detection architecture additionally enables polarization multiplexing, where independent data streams are transmitted on orthogonal polarization states and subsequently separated through digital processing. Dual-polarization coherent receivers produce four electrical outputs (X-pol I/Q and Y-pol I/Q), effectively doubling the transmission capacity without requiring additional optical spectrum [6].

The importance of coherent detection for modern optical communications stems from these combined advantages. However, historically direct detection has won out against coherent detection due to its relative simplicity. Advances in DSP, and importantly, high-speed analogue-to-digital converters (ADCs) solved many of the impracticalities associated with coherent detection [6, 7]. While direct detection remains viable for cost-sensitive short-reach applications where fibre impairments are minimal, long-haul and high-capacity systems increasingly rely on coherent detection.

2.1.2 The Principles of Coherent Detection

There is some variation in the specifics of coherent detection types, but they can be broadly categorised into heterodyne and homodyne. Heterodyne detection operates with a deliberate frequency offset between the LO and signal carrier, producing an electrical output at an intermediate frequency (IF). While this relaxes frequency-stability requirements, the IF must be significantly larger than the signal bandwidth, doubling the required electrical bandwidth [6]. Homodyne detection necessitates the LO frequency equal (or be close to) the signal carrier, down-converting directly to baseband. Classically, homodyne receivers required an optical phase-locking loop (OPLL) to track carrier phase. This proved impractical due to large loop delays and the instability of semiconductor laser phase [6]. The introduction of the 'phase-diversity homodyne receiver' which uses a 90° optical hybrid, resolved this limitation by measuring both in-phase and quadrature components simultaneously, deferring phase recovery to the digital domain [6]. This architecture (referred to as intradyne) tolerates a small frequency offset between signal and LO and corrects for it digitally. This is bedrock of all modern commercial coherent optical systems [6].

The fundamental principle of coherent detection is the mixing between the received signal field and a local oscillator. Optical transmission gives us the benefit of the speed of light, however signals at these frequencies, which are in the terahertz, are far beyond the bandwidth of any electrical system. Coherent mixing effectively down-converts the optical signals into baseband electrical frequencies that can be sampled and processed digitally.

The incoming signal (in the case of dual polarisation) is split into two polarisation planes (X and Y) by a polarisation beam splitter (PBS). The local oscillator is also split, but usually by power rather than polarisation, by a coupler. Mixing of the signals happens within an 'optical hybrid' circuit. A 90° optical hybrid performs mixing by splitting both the signal and LO into multiple paths, combining them at different phase offsets, and directing the mixed outputs to balanced photodetector pairs. This generates 4 outputs (2 differentially paired outputs), corresponding to in-phase (I) and quadrature (Q) components. For a dual-polarization receiver, these outputs comprise $XI(t)$, $XQ(t)$, $YI(t)$, and $YQ(t)$, representing the complete four-dimensional signal space required to recover polarization-multiplexed transmission. The benefit of using a differential detection scheme is that DC photocurrent contributions and common-mode noise are cancelled out. This effectively doubles the electrical signal power while suppressing noise components, improving the receiver sensitivity [6].

The balanced photodetector photocurrent for each channel follows the relationship:

$$I(t) = I_1(t) - I_2(t) = 2R\sqrt{P_s(t)P_\ell} \cos\{\omega_{IF}t + \theta_{sig}(t) - \theta_\ell(t)\} \quad (2.1)$$

[6] where R is the photodetector responsivity, $P_s(t)$ and P_ℓ are the signal and LO powers respectively, $\omega_{IF} = |\omega_s - \omega_\ell|$ is the intermediate frequency, and $\theta_{sig}(t)$ and $\theta_\ell(t)$ are the phases of the signal and LO. This expression demonstrates the optical amplification effect inherent to coherent detection: the photocurrent scales with $\sqrt{P_\ell}$, hence increasing the LO power provides gain for the detected signal [6].

The beat frequency generated by differences between the signal and LO frequencies manifests as the

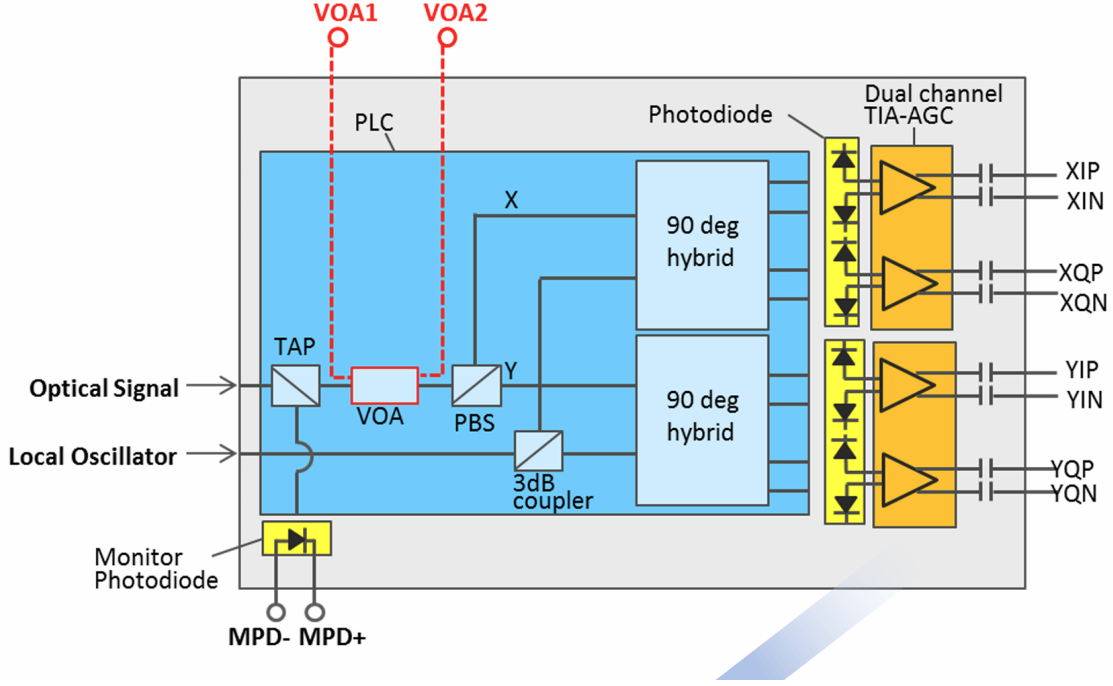


Figure 2.1: Internal structure of FIM24725 [1]

intermediate frequency, which in linear frequency units is given by:

$$\Delta f = \frac{|\omega_s - \omega_\ell|}{2\pi} \quad (2.2)$$

For optical carriers, the relationship between wavelength offset and frequency offset can be obtained from the differential of: $f = c/\lambda$:

$$\Delta f \approx \frac{c}{\lambda^2} \cdot \Delta \lambda \quad (2.3)$$

where c is the speed of light, λ is the optical wavelength, and $\Delta \lambda$ represents the wavelength offset between signal and LO. Wavelength detuning on the order of nanometres at telecommunication wavelengths translates to electrical beat frequencies in the gigahertz range.

2.1.3 The Fujitsu FIM24725 Micro-Integrated Coherent Receiver

The Fujitsu FIM24725 micro-integrated coherent receiver (micro-ICR) is a highly integrated intradyne coherent detector. It integrates a PBS, variable optical attenuator (VOA), monitoring photodiode (MPD), two 90° optical hybrids, eight photodiodes, and four differential transimpedance amplifiers (TIAs) into a single compact module.

This internal structure (Figure 2.1) implements complete dual-polarisation coherent reception. The PBS demultiplexes the signal into X and Y components which are orthogonal to each other. The LO is split by a 3dB coupler. These separate components are each mixed in an optical hybrid producing four optical outputs (I+, I-, Q+, Q-), which illuminate photodetector pairs. The resulting photocurrents are converted into voltages by the TIAs with some gain applied: either automatic gain control (AGC) or manual gain control (MGC).

The receiver operates across the C-band, covering 191.35 THz to 196.20 THz (approximately 1528

nm to 1567 nm). The signal input port accepts average optical power from -18 dBm to 0 dBm, with the LO requiring between +3 dBm and +13 dBm. The built-in variable optical attenuator (VOA) provides up to 10 dB of attenuation on the signal path, with a response time under 20 ms.

The receiver supports symbol rates up to 32 GBaud with a small-signal 3 dB bandwidth of 22 GHz and a low-frequency cutoff at 100 kHz. The differential output swing is adjustable from 300 to 900 mVppd. Phase error between I and Q channels is within ± 5 degrees, and channel skew is 10 ps max [8]. The significance of this level of integration is that a single package replaces what would otherwise be a bunch of discrete optical and electrical components, but it creates a device with multiple control inputs, strict sequencing requirements, and a need for reliable monitoring during operation.

2.2 Laboratory Instrument Control

Instrument control lives in several distinct paradigms, each offering trade-offs between flexibility, safety, and complexity. The understanding of these approaches and their limitations provide context for architectural decision made in this project.

2.2.1 SCPI and Command-Based Instrument Controls

The Standard Commands for Programmable Instruments (SCPI) specification, formalised by the SCPI Consortium in 1990 [9], defines a standardised ASCII command syntax for controlling test and measurement equipment. SCPI commands follow a hierarchical tree structure: for example, *SOURce:VOLTage 3.3* to set an output voltage, or *MEASure:CURRent?* to query a measured current. They are transport agnostic i.e. they can be sent over any transport interface such as USB, Ethernet, etc. The Multicomp Pro MP710508 power supplies used in this project employ a SCPI-like (not actual standard SCPI, see A1.1) UDP command interface for remote control.

SCPI provides a common language for communicating with instruments, but it's fundamentally a command protocol and not a control framework. It does not conceptualise sequencing, preconditions, or safety. This must be implemented entirely in the calling code. In practice this usually manifests in ad-hoc scripting that issue sequences of commands to automate repetitive tasks. These are typically use-case-specific, rarely version controlled, and knowledge is encoded only implicitly through the order of commands. When a script is shared or revisited after long periods, the rationale behind the script may be lost.

2.2.2 Large-Scale Control Frameworks: EPICS

On the total other end of the complexity spectrum, the Experimental Physics and Industrial Control System (EPICS) is a comprehensive, open-source software framework developed originally at Los Alamos National Laboratory and Argonne National Laboratory for controlling large scientific facilities [10]. EPICS is widely deployed, it is used and developed by at least 43 large science institutions worldwide, from things like synchrotrons to the International Thermonuclear Experimental Reactor (ITER) in France [11, 12].

It is built around the concept of process variables (PVs) which are named, typed, network-accessible

data points that represent instrument parameters. An Input/Output Controller (IOC) hosts device drivers and publishes PVs over a custom network protocol called Channel Access (or its successor, pvAccess). Client applications subscribe to PVs to issue control commands. The framework provides alarms, access control, and archiving, and can enforce interlocks by linking PVs together such that a condition is a prerequisite for another action [10, 13].

While powerful, EPICS demands substantial infrastructure overhead. Deploying even minimal instances requires defining a database of records in a domain-specific configuration language, configuring the Channel Access network layer, and building or adapting device support modules for each instrument type. It assumes facility-scale and dedicated control engineers. For a single instrument deployment with minimal hardware, the configuration effort required by EPICS would far exceed the complexity of the problem it solves.

2.2.3 Middleground: GUI-Based Frameworks like LabVIEW

National Instruments' LabVIEW is arguably the most widely used instrument control platform in academic and industrial laboratories. Users can construct virtual instruments by wiring together functional blocks. This provides an intuitive interface for data acquisition, signal processing, and instrument communication. LabVIEW includes extensive driver libraries (through the IVI and VISA standards) and integrates tightly with National Instruments' own hardware ecosystem [14].

LabVIEW can certainly be used to build stateful, safety-aware control systems. Its Queued Message Handler pattern and Actor Framework provide mechanisms for state management, message passing, and component lifecycle control. However, several characteristics of the platform make it a poor fit for the specific deployment model required by this project.

LabVIEW's model assumes a desktop GUI environment with a licensed operator present. Its proprietary per-seat licensing and dependence on the LabVIEW Runtime Engine make containerised or server-based deployment impractical [15]. Additionally, the project's instruments use non-NI hardware with a UDP command interface. The abstraction layers that LabVIEW provides (VISA, IVI drivers) offer no advantage over direct socket communication in a general-purpose language.

2.2.4 Other Modern Lightweight Approaches

More recently, the Python scientific ecosystem has brought about many instrument control libraries such as PyVISA [16], which provides a wrapper around the VISA (Virtual Instrument Software Architecture) standard. This allows SCPI commands to be sent over USB, serial, Ethernet, etc. from Python scripts. InstrumentKit [17] goes further, offering object-oriented abstractions for specific instrument families.

These tools are an improvement over raw scripting. They provide cleaner APIs, better error handling, and in some cases instrument-agnostic abstractions. However, they remain fundamentally instrument communication libraries, not instrument control systems. A PyVISA script can set a voltage out of bounds or enable outputs in the wrong order just as easily as a raw SCPI script can; it simply does so with more readable syntax.

2.2.5 The Gap

The approaches reviewed above span a wide range of capability and complexity, but none directly addresses the specific requirements of this project: safe, state-aware, remotely accessible control of an instrument with strict operational constraints, deployable without the specialist controls infrastructure and configuration overhead associated with facility-scale frameworks.

This project occupies the gap in approaches. It requires a system that is lightweight enough to be developed and deployed by a single engineer, yet structured enough to enforce safety-critical sequencing and parameter validation as architectural guarantees rather than ad-hoc scripting conventions.

2.3 Software Concepts

2.3.1 Finite State Machines

A finite state machine (FSM) is an event-driven model of a system or process which exists in one of a finite number of states at any given time. Transitions between states occur only in response to defined events [18]. FSMs are widely used in embedded systems and process control to model device lifecycles. For example, a power supply unit might be modelled as having states such as OFF, INITIALISING, ACTIVE, and FAULT. The value of an FSM in a safety-critical context is that potentially harmful operations become structurally impossible. If a transition from OFF directly to ACTIVE is not defined it cannot occur. The system described in Chapter 3 uses a state machine to manage the coherent receiver's lifecycle and gate all control operations by the current device state.

2.3.2 Hardware Abstraction Layers

A Hardware Abstraction Layer is typically defined as something like a software layer that sits between the physical hardware of a computer and the software running it [19]. The operating system would communicate with the HAL to activate the necessary hardware. In this project it refers to the software layer which sits between the MP710508 power supplies and Arduino Uno R3. The purpose is to decouple application logic from the specifics of physical hardware communication (e.g. `set_voltage(3.3)`).

There are two practical benefits from using a HAL: the application logic is independent of the communication transport. If the interface changes from UDP (User Datagram Protocol) to serial, only the HAL implementation needs to change. Second, the HAL can be mocked with a simulated implementation of hardware, allowing software development and hardware integration to proceed in parallel.

2.3.3 Separation of Concerns and Layered Architecture

"Separation of Concerns (SoC) is a fundamental principle in software engineering and design aimed at breaking down complex systems into smaller, more manageable parts. The goal is to organize a system's components in a way that each part addresses a

single concern, or a cohesive aspect of functionality, rather than mixing multiple concerns together. This approach enhances modularity, maintainability, and scalability of software systems.” [20]

In the context of instrument control, a typical layering might place a hardware layer communication at the bottom, sequencing logic in the middle, and a user-facing interface at the top. These are important tenets for a safety-critical system, but also a generally appreciable software design principle. It localises enforcement, makes debugging easier, and clearly defines paths through a system: a UI button press, an API call, or an automated script all pass through the same safety checks.

2.3.4 REST APIs and Web-Based Interfaces

A REST (Representational State Transfer) API (application Programming Interface) is a widely adopted pattern for exposing software functionality over HTTP (Hyper-Text Transfer Protocol) [21]. Clients send HTTP requests to defined endpoints and receive structured responses, usually in JSON (Java Script Object Notation) format. They are platform and programming-language agnostic, which makes them well suited to a laboratory where control interfaces may be accessed from different devices and operating systems.

Web-based interfaces complement REST APIs beautifully, by serving interactive control through a standard browser. This eliminates the need for custom client software to be installed on each user’s machine. For remote lab access, this is particularly useful as a user could monitor and control equipment using the same browser they would for any other web application, even from their phone.

2.3.5 Containerisation

Containerisation is a deployment technique in which an application and all of its dependencies are packaged together into a self-contained ‘image’ that runs isolated on any compatible host [22]. Docker is the most widely used containerisation platform and provides a mechanism for defining and building containerised images.

In the context of this project it addresses a persistent practical problem: making software run identically on different platforms, from the development platform, a laboratory server, or an embedded deployment target. A containerised deployment is reproducible by construction.

3 Design and Implementation

3.1 Design Requirements

3.1.1 Characterising the FIM24725

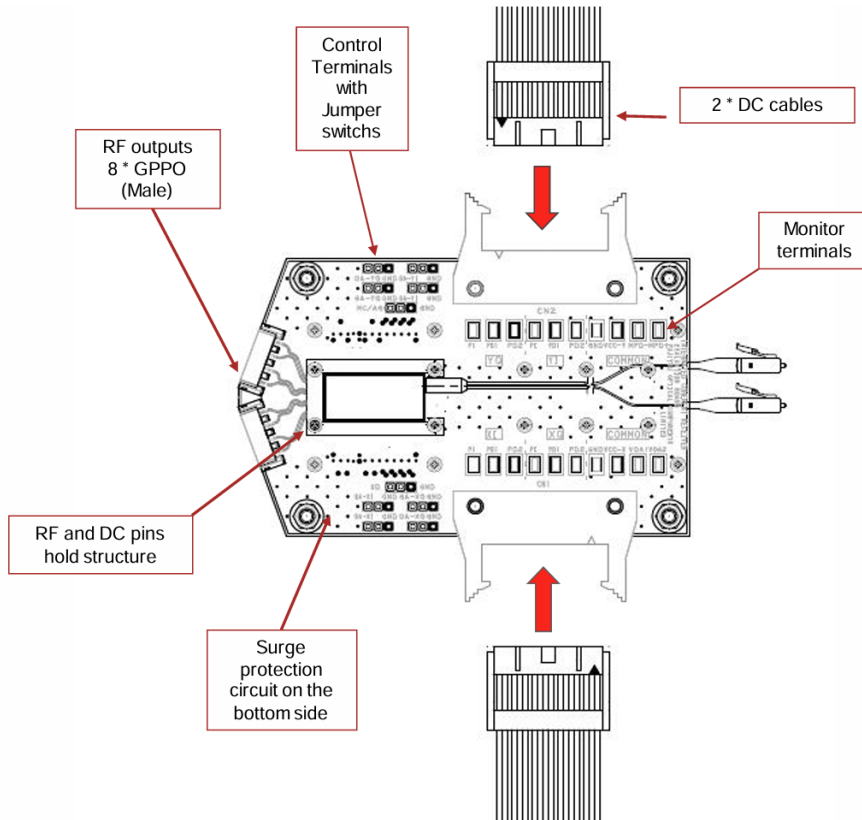


Figure 3.1: Annotated layout of the FIM24725 showing key components and interfaces [1].

Figure 3.1 details the interfaces present on the FIM24725. Every input and output, and their respective operating envelopes for the FIM24725 was formally defined in accordance with its datasheet [8] and supporting application notes [1]. The tables below serve to condense data sheet information into a single source of truth to ensure that ensures that the device is operated within its specified electrical limits. Formally tabulating these interfaces in terms of signal outputs (labelled as RF outputs 8 * GPPO (Male)) and control inputs/outputs (labelled as 2 * DC Cables) provides the basis for system integration.

Table 3.1: FIM24725 DC pin functions and operating ranges.

Pin	Name	Function	Type	Voltage Range	Current	Notes
<i>Reserved</i>						
1	RFU	Reserved for Future Use	–	–	–	–
2	RFU	Reserved for Future Use	–	–	–	–
33	RFU	Reserved for Future Use	–	–	–	–
34	RFU	Reserved for Future Use	–	–	–	–
<i>Digital control pins</i>						
3	MC/AGC	Gain control mode selection	Input	0–0.8 V (MGC) 2– V_{cc} (AGC)	–	1 M Ω pull-up to V_{cc} (floats to AGC). Active-LOW shutdown.
32	SD	Shutdown	Input	0–0.8 V (SD active) 2– V_{cc} (SD inactive)	–	1 M Ω pull-up (floats to V_{cc} .)
<i>Analog control pins — gain adjust (MGC mode)</i>						
11	GA-YI	Gain Adjust Y-I	Input	0– V_{cc}	–	Ignored in AGC mode. Influences output swing.
16	GA-YQ	Gain Adjust Y-Q	Input	0– V_{cc}	–	As above.
19	GA-XI	Gain Adjust X-I	Input	0– V_{cc}	–	As above.
24	GA-XQ	Gain Adjust X-Q	Input	0– V_{cc}	–	As above.
<i>Analog control pins — output amplitude (AGC mode)</i>						
12	OA-YI	Output Amplitude Adjust Y-I	Input	0.5–2.0 V	–	115 k Ω pull-up to 1.75 V. Influences output swing.
15	OA-YQ	Output Amplitude Adjust Y-Q	Input	0.5–2.0 V	–	As above.
20	OA-XI	Output Amplitude Adjust X-I	Input	0.5–2.0 V	–	As above.
23	OA-XQ	Output Amplitude Adjust X-Q	Input	0.5–2.0 V	–	As above.
<i>VOA control</i>						
31	VOA1	VOA control voltage	Input	0–4.8 V	≤ 80 mA	Max attenuation 10 dB. Abs. max 5.2 V.
30	VOA2	VOA ground	Ground	0 V	–	Hard-tied to GND.
<i>Monitor / telemetry pins</i>						
4	MPD+	Monitor Photodiode Cathode	Output	0–2 V	–	Responsivity 0.05 A/W. LO-input isolation 45 dB.
5	MPD–	Monitor Photodiode Anode	Output	0–2 V	–	Differential pair with MPD+.
10	PI-YI	Peak Indicator Y-I	Output	0–2 V	–	0 V = no signal; 2 V = clipping.
17	PI-YQ	Peak Indicator Y-Q	Output	0–2 V	–	As above.
18	PI-XI	Peak Indicator X-I	Output	0–2 V	–	As above.
25	PI-XQ	Peak Indicator X-Q	Output	0–2 V	–	As above.
<i>Power supply pins — amplifier (V_{cc})</i>						
13	VCC_Y	Amplifier supply (YI & YQ)	Power	3.135–3.465 V	typ. 360 mA max 400 mA	Abs. max 4.5 V. Power-down before V_{PD} .
22	VCC_X	Amplifier supply (XI & XQ)	Power	3.135–3.465 V	typ. 360 mA max 400 mA	As above.
<i>Power supply pins — photodiode (V_{PD})</i>						
6	VPD_YIn	PD supply Y-In	Power	4.75–5.25 V	–	Abs. max reverse 7 V. Power-up before V_{cc} .
7	VPD_YIp	PD supply Y-Ip	Power	4.75–5.25 V	–	As above.
8	VPD_YQp	PD supply Y-Qp	Power	4.75–5.25 V	–	As above.
9	VPD_YQn	PD supply Y-Qn	Power	4.75–5.25 V	–	As above.
26	VPD_XIn	PD supply X-In	Power	4.75–5.25 V	–	As above.
27	VPD_XIp	PD supply X-Ip	Power	4.75–5.25 V	–	As above.
28	VPD_XQp	PD supply X-Qp	Power	4.75–5.25 V	–	As above.
29	VPD_XQn	PD supply X-Qn	Power	4.75–5.25 V	–	As above.
<i>Ground</i>						
14	GND	Signal ground	Ground	0 V	–	–
21	GND	Signal ground	Ground	0 V	–	–

Table 3.2: FIM24725 RF signal output pins.

Pin	Name	Function	Type	Notes
1	XIp	X In-phase positive	Output	Differential pair. AC coupled. 300–900 mV _{ppd} swing.
2	XIn	X In-phase negative	Output	
3	XQp	X Quadrature positive	Output	As above.
4	XQn	X Quadrature negative	Output	
5	YIp	Y In-phase positive	Output	As above.
6	YIn	Y In-phase negative	Output	
7	YQp	Y Quadrature positive	Output	As above.
8	YQn	Y Quadrature negative	Output	

A few observations from table 3.1 are worth drawing out, as they inform some design choices that follow.

The SD and MC/AGC both have internal pull-ups such that when left floating settle to SD inactive (output enabled) and AGC mode respectively. These defaults are sensible defaults for a minimal control set-up that aims to minimise control lines, however as we are investing in building a control system for the receiver, we may as well explicitly set them during start-up and operation. Disabling output initially means that the receiver can settle before worrying about outputs, it also serves as a flag for when optical input can be applied, and can protect downstream equipment from damage. It is worth noting, however, that SD controls only the output buffer stage and offers no protection to the internal photodiode-TIA interface nor disconnects the input path from the photodiodes. In the case that optical input is applied before the receiver is sequenced the photodiodes would act in photovoltaic mode applying an undefined potential to TIA inputs. If the startup sequence was then followed: applying biasing to photodiodes, a photocurrent would flow through the TIAs while they are unpowered which is potentially damaging though unlikely at the input signal power limits for the receiver (around 1 mA at worst case from 13 dBm LO with 0.05 A/W responsivity) but should not be relied upon as ESD structures are not intended for sustained current. All this is to say that SD protects downstream equipment, but cannot protect the receiver from optical input prior to startup. This means signal input must be managed manually by users in the lab and must only be applied when the receiver is ready, which aligns with Fujitsu’s guidance. [1].

Although the FIM24725 presents 34 DC pins, many can be electrically grouped. All 8 VPD pins require the same 5 V bias supply, both VCC pins require the same 3.3 V supply, VOA control and ground form a single analogue input, and OA and GA pins could be grouped by polarisation if necessary. This is significant because it reduces the minimum number of controllable channels the system requires. This is discussed further in section 3.2, where available hardware is mapped to these groupings.

Finally, there is a meaningful buffer between operating ranges and the specified absolute maximum. For example, VCC operates within 3.135–3.465 V but the absolute maximum is 4.5 V. This makes over-voltage and over-current protections viable and useful.

3.1.2 Safety-Critical Sequencing Constraints: Start-up & Shutdown Algorithms

The general recommended power-up and power-down sequences for the receiver are well defined in the general application notes (see Appendix A1.2). Critically, during startup the photodiodes must be biased before power is supplied to the TIAs or there will be permanent hardware damage. The

inverse is true for shutdown. The manufacturer guidelines are sufficient for the critical path, but they assume a cooperative manual operator and a clean initial state. Additional provisions are made in the case of this control system:

1. **Verify Initial State:** Confirm communication with external devices, and ensure PSU outputs are all disabled. This prevents a startup attempt from an unknown or partially-energised state.
2. **Lock PSU panels:** Prevent a user manually adjusting through the physical controls, which would bypass the software safety layer entirely.
3. **Protection programming:** OVP and OCP thresholds must be programmed and enabled on every channel prior to enabling, as discussed in Section 3.1.1.
4. **Predefined resultant state:** Control signals (GA, OA, VOA) must be set to known safe default values. This ensures the receiver starts in the same manner every time and does not leave control values floating.
5. **Settling time.** A brief delay should be allowed so supply voltages and the TIA operating points can stabilise.
6. **Operational validation.** After enabling the output, the system should read the peak indicator (PI) and MPD telemetry to confirm that the receiver is producing output. This provides a basic end-to-end verification that the startup sequence completed successfully.

The following high-level algorithms describe the startup and shutdown procedures as implemented by the control system. Detailed software implementation is discussed in Section 3.3.2.

High-level startup algorithm:

1. Check system state: require OFF.
2. Verify communication with external devices.
3. Disable receiver output (SD active) and set gain mode (AGC by default).
4. Lock PSU front panels.
5. Program OVP and OCP protections on all rails.
6. Enable photodiode bias supply. Verify readback.
7. Enable amplifier supply. Verify readback.
8. Set default control values (VOA, OA, GA).
9. Allow settling time.
10. Enable receiver output (SD inactive).
11. Read peak indicators to validate operation.
12. Transition system state to READY.

High-level shutdown algorithm:

1. Disable receiver output (SD active).
2. Disable control signals (GA, OA, VOA).
3. Disable amplifier supply. Verify readback.
4. Disable photodiode bias supply. Verify readback.
5. Disable all remaining PSU outputs.

6. Transition system state to OFF.
7. Unlock PSU front panels.

3.1.3 Functional Requirements

The initial project brief came from loose discussions with the project supervisor, which may be summarised as “the implementation of a graphical interface for controlling the coherent receiver’s voltage sources from an external computer, with the ability to set bias points, enforcement of maximum and minimum operating limits, and automated power-up and power-down sequences”. The broader system scope, architecture, and implementation details were not rigidly specified, but were developed broadly as part of the project plan. Informed by the device characterisation, analysis of what would constitute ‘safe operation’ in this context, and the consideration of other generally useful features in consultation with lab staff, the functional requirements in Tables 3.3 - 3.6 were derived:

Table 3.3: FR1 — Power sequencing and lifecycle management.

ID	Requirement
FR1.1	The system shall power-up and power-down the FIM24725 in accordance with section 3.1.2.
FR1.2	Startup and shutdown shall be user-initiated actions; system shall not power the module automatically on launch.
FR1.3	OVP and OCP shall be programmed on every rail before any output is enabled.
FR1.4	PSU front panels shall be locked to prevent manual adjustment that would bypass software safety controls.
FR1.5	The system shall implement a state machine governing the receiver lifecycle. Operations shall only be permitted when the system is in the appropriate state.
FR1.6	Fault information (message, timestamp, affected component) shall be reported to the operator.
FR1.7	A fault condition shall be reachable from any state and may trigger a best effort emergency shutdown.

Table 3.4: FR2 — Input control.

ID	Requirement
FR2.1	The system shall allow the operator to set VOA, OA, and GA control voltages at runtime.
FR2.2	Values outside the permitted range shall be rejected before reaching the hardware.
FR2.3	The system shall support runtime switching between AGC and MGC modes.

Table 3.5: FR3 — Telemetry and monitoring.

ID	Requirement
FR3.1	The system shall acquire and display PI and MPD data at a rate sufficient to observe system behaviour.
FR3.2	All commands, state transitions, setpoints, readback values, and faults shall be logged with timestamps.

Table 3.6: FR4 — User interface.

ID	Requirement
FR4.1	The system shall provide a web-based graphical interface for all control, monitoring, and log-viewing functions.
FR4.2	The interface shall reflect the current system state and reject inputs that are not valid.

3.1.4 Non-Functional Requirements

Beyond what the system must do, a number of requirements govern how it must behave. These were derived from the laboratory context and practical constraints of the project given in Tables 3.7 - 3.9.

Table 3.7: NFR1 — Safety, reliability, concurrency.

ID	Requirement
NFR1.1	Loss of communication (avoiding transient delays) with any external hardware shall result in a fault and an attempt to transition to a safe state.
NFR1.2	Hardware commands shall use a write-then-verify pattern.
NFR1.3	Given the concurrent access nature of web-apps commands must be serialised.
NFR1.4	Commands should be executed in the order they are requested. Telemetry acquisition shall not block or delay safety-critical operations.

Table 3.8: NFR2 — Maintainability, Usability and extensibility.

ID	Requirement
NFR2.1	The architecture shall be layered, operating under separation of concerns.
NFR2.2	Hardware communication layers shall impose no policy, allowing reuse with other systems.
NFR2.3	The system shall survive host power cycles and restart automatically without manual intervention.
NFR2.4	The full control interface shall be accessible from a standard web browser with no client-side installation necessary.
NFR2.5	The software shall be platform agnostic: migration to a different host shall be undemanding.
NFR2.6	Environment-specific parameters (network addresses, device paths) shall be configurable without code changes.
NFR2.7	A researcher unfamiliar with the FIM24725's internal constraints must be able to operate the receiver safely.

Table 3.9: NFR3 — Security and Access.

ID	Requirement
NFR3.1	Access control shall be network level through existing lab VPN infrastructure.
NFR3.2	The control interface shall not be accessible to unauthorised users.

3.2 Hardware Platform

Given the short timespan of the project, the system was built from hardware that was already readily available in the CONNECT lab or could be readily sourced without procurement delays. This section describes each hardware component and the rationale for its selection, and the constraints that emerge from their selection which directly inform the software architecture described in Section 3.3.

3.2.1 Power Supply Units (PSUs): Multicomp Pro MP710508



Figure 3.2: Multicomp Pro MP710508 (image reproduced from [2])

The laboratory had two Multicomp Pro MP710508 programmable bench power supplies available and set aside for this project. Each power supply provides four independently controllable channels, each with programmable voltage and current setpoints, OVP and OCP protections. They operate a simple UDP-over-IP communication protocol which takes SCPI-like commands (see Appendix A1.1) which enables programmatic control from any networked host. They also allow the properties of each channel to be queried, which ensures commands have been enacted in the physical system, and provides a mechanism for monitoring parameters like current draw to facilitate fault detection.

While a PSU may seem like an unconventional choice for delivering analogue control voltages compared to a digital-to-analogue converter (DAC), there are several advantages in this case. Firstly,

is the fact that the PSUs were available and set aside for the project, requiring no lead time. Second, a DAC provides no inherent OVP or OCP protection, which is available from the MP710508, as required by FR1.3. Thirdly, the MP710508 enables a write-then-verify pattern through the ability to query channel properties (as required by NFR1.2) that a DAC does not. Lastly, it can serve as both a power supply for photodiode biasing and TIA power and an analogue controller. A DAC would still require a separate power source capable of delivering the hundreds of milliamps to the TIAs.

The main trade-off is that the two MP710508s leave only eight channels to play with. This necessitates that control be collapsed to a maximum of eight channels. These mappings are herein referred to as 'rails'.

Table 3.10: PSU 1 Control Rail Mapping: Amplifier supply, diode bias supply, VOA.

Ch	Rail Name	Desc.	FIM24725 Pin	VSET	ISET	OVP	OCP
1	VCC_3V3	Amplifiers power supply (X and Y)	VCC_Y (13), VCC_X (22)	3.300V	0.500A	3.600V	0.700A
2	VPD_5V0	Photodiode power supplies	VPD_Y* (6–9), VPD_X* (26–29)	5.000V	0.050A	5.500V	0.150A
3	VOA_CTRL	Variable optical attenuator	VOA1 (31), ref. to VOA2 (30 = GND)	0–4.800V	0.080A	5.000V	0.100A

Table 3.11: PSU 2 Control Rail Mapping: Gain adjust and offset adjust.

Ch	Rail Name	Desc.	FIM24725 Pin	VSET	ISET	OVP	OCP
1	GA_X	Gain adjust (X polarisation)	GA_XI (21), GA_XQ (20)	0–3.3V	0.020A	3.600V	0.020A
2	GA_Y	Gain adjust (Y polarisation)	GA_YI (14), GA_YQ (15)	0–3.3V	0.020A	3.600V	0.020A
3	OA_X	Offset adjust (X polarisation)	OA_X (19)	0.5–2V	0.020A	3.600V	0.020A
4	OA_Y	Offset adjust (Y polarisation)	OA_Y (16)	0.5–2V	0.020A	3.600V	0.020A

In reference to Tables 3.10 and 3.11, rail VPD_5V0 drives all eight photodiode bias pins from a single PSU channel as all eight pins require the same configuration. The same logic applies to rail VCC_3V3 which feeds both VCC_X and VCC_Y from a single channel. Although the FIM24725 has four independent GA and four independent OA pins, the channel constraint forces them to be paired by polarisation. The trade-off is that independent per-quadrature gain control is not possible however, in typical operation, I and Q within a polarisation see similar signal levels, so independent control is a refinement rather than a necessity.

The VSET, ISET, OVP, and OCP values in Tables 3.10 and 3.11 were chosen with reference to the operating envelope defined in Section 3.1.1. For VCC_3V3, for example, the OVP is set at 3.600 V which is above the operating maximum of 3.465 V but well below the 4.5 V absolute maximum. The current limit (ISET) is set at 0.500 A, above the rated maximum amplifier supply current of 400 mA to permit normal operation, while the OCP at 0.700 A provides a further backstop against gross

overcurrent faults.

The MP710508's UDP interface has one notable operational constraint: responses are not tagged, and it does not support multiple outstanding requests. Therefore, the PSU driver must serialise all commands as discussed in Section 3.3.1.

3.2.2 Microcontroller (MCU): Arduino Uno R3

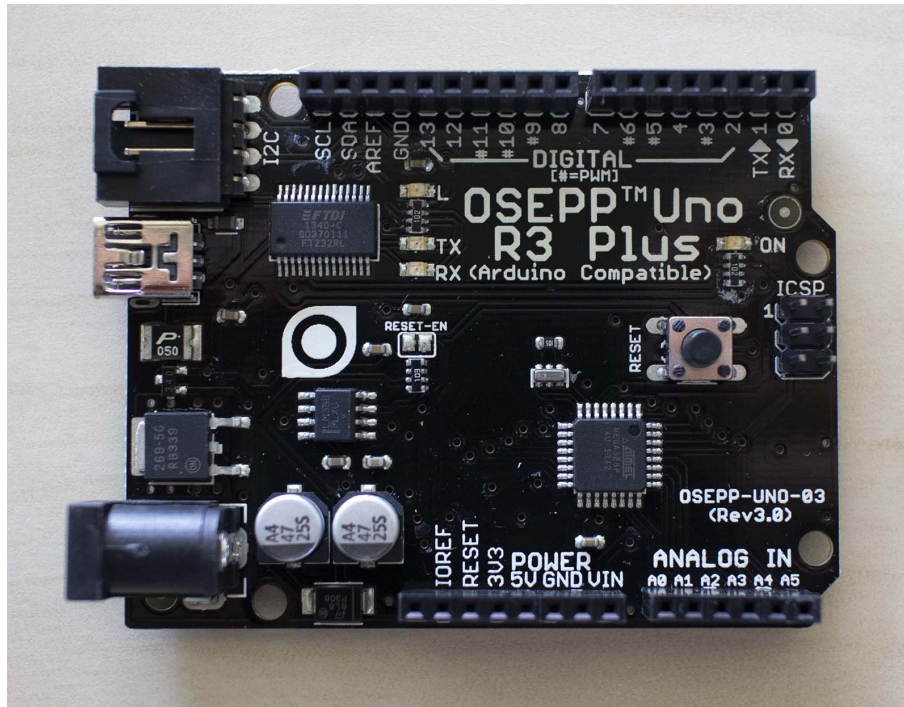


Figure 3.3: OSEPP Arduino Uno R3 (image reproduced from [3])

The PSUs provide analogue voltage control and power, but have no digital I/O capacity. They cannot read the five analogue monitor outputs (four PI channels and MPD) and should not provide the two digital control signals (SD and MC/AGC). Moreover, all available channels bar one has already been allocated. A peripheral capable of digital output and analogue-to-digital conversion is therefore needed to bridge this gap.

An OSEPP Arduino Uno R3, available in the laboratory, was selected to fill this role. The Uno provides 14 digital I/O pins and six 10-bit ADC channels, which is more than sufficient. It was chosen over other microcontroller platforms primarily because it was on hand and met the functional requirements without procurement delay. It also has the benefit of Arduino IDE support and large library support which makes writing firmware for it simple.

Unfortunately, the Uno does not support Ethernet natively. Ideally, all hardware in the system would communicate with the host via IP, simplifying the topology of the network. An Ethernet shield (such as the W5500) or an Ethernet-native microcontroller board would achieve this, but either option would have required procurement time as none were available in college. The Arduino is therefore addressed over UART serial. Under Linux, the device appears as a standard serial port (`/dev/ttyUSBx`). This unfortunately means the Arduino must be connected to the host by USB cable. An Ethernet-capable board remains a viable future upgrade path that would require no

changes to the software interface beyond substituting the serial transport for a UDP transport, since the application-level protocol is independent of the physical layer (NFR2.2)

The original project plan specified deployment to the laboratory server infrastructure, which would have to be accessed over the network. In this case, the digital and analogue signals would need to reach the server by some networked means: either an Ethernet capable microcontroller or network module for a microcontroller. The deployment target shifted over the course of the project from the lab server to some co-located host after the system administrator for the lab servers went on leave. This meant that USB connection was less of an immediate issue, however, to maintain NFR 2.6, it would be preferable to upgrade to an Ethernet capable microcontroller although this transition was not achievable within the project timeframe and constraints.

A co-located host still does not solve the aforementioned problems, as a general-purpose computer is incapable of GPIO and has no built in ADC, and trying to drive physical hardware from a containerised deployment would complicate the model targeted by NFR2.5.

All of this is to say: a microcontroller capable of some GPIO and reading ADC was a necessary piece of hardware. The co-located host/computer (discussed next) would not remove the need for it. Time-permitting it would have been a better solution to use a network-capable controller, but this is possible with minimal changes in the future (for example in a laboratory server deployment).

Table 3.12: MCU Control and Read Mapping: Digital control and analogue monitoring.

MCU Signal	Desc.	MCU Pin	FIM24725 Pin	Direction
SD	Shutdown (enable)	D2	SD (32)	TX
MC/AGC	Manual vs automatic gain control	D3	MC/ AGC (3)	TX
PI	Peak indicator monitoring	A0–A3	PI-XI (18), PI-XQ (25), PI-YI (10), PI-YQ (17)	RX
MPD	Monitor photodiode anode and cathode	A4, A5	MPD+ (4), MPD– (5)	RX

Table 3.12 shows the pin assignments. All analogue read pins are wired with external pull-down resistors to stop floating behaviour. Digital write pins (SD and MC/AGC) are wired with external voltage dividers to bring the 5 V Arduino logic level within the 3.3 V FIM24725 logic range. The ADC reference voltage is set to 3.3 V by connecting the Arduino's 3.3 V regulator output to the AREF pin. With a 10-bit ADC and a 3.3 V reference, the conversion yields approximately 3.2 mV resolution. This is adequate for real-time monitoring: observing whether PI values are near zero (no signal), mid-range (normal operation), or near 2 V (clipping), and for simple gain maximisation. It may not be sufficient for precision characterisation measurements: the PI and MPD readings serve as operational indicators, not calibrated measurements.

3.2.3 Service Deployment Target: Raspberry Pi 3 Model B

As aforementioned the original project plan specified deployment to the laboratory's shared server infrastructure. In practice, the system administrator responsible for these servers was unavailable during the project period, and deploying on shared infrastructure would have required coordination



Figure 3.4: Raspberry Pi 3 Model B (image reproduced from [4])

incompatible with the project timeline. A Raspberry Pi 3 Model B, available in the laboratory, was adopted as the deployment target instead.

This decision had unanticipated benefit: hosting the control system on a dedicated system co-located with the MCU and receiver transforms the set-up into a self-contained portable unit. The system requires only a power connection and a network connection to operate. This is attractive for a shared laboratory instrument that may be physically relocated. A user need only connect power, Ethernet, and optical fibres; no software installation or configuration is required on their local machines (NFR2.4, NFR2.5).

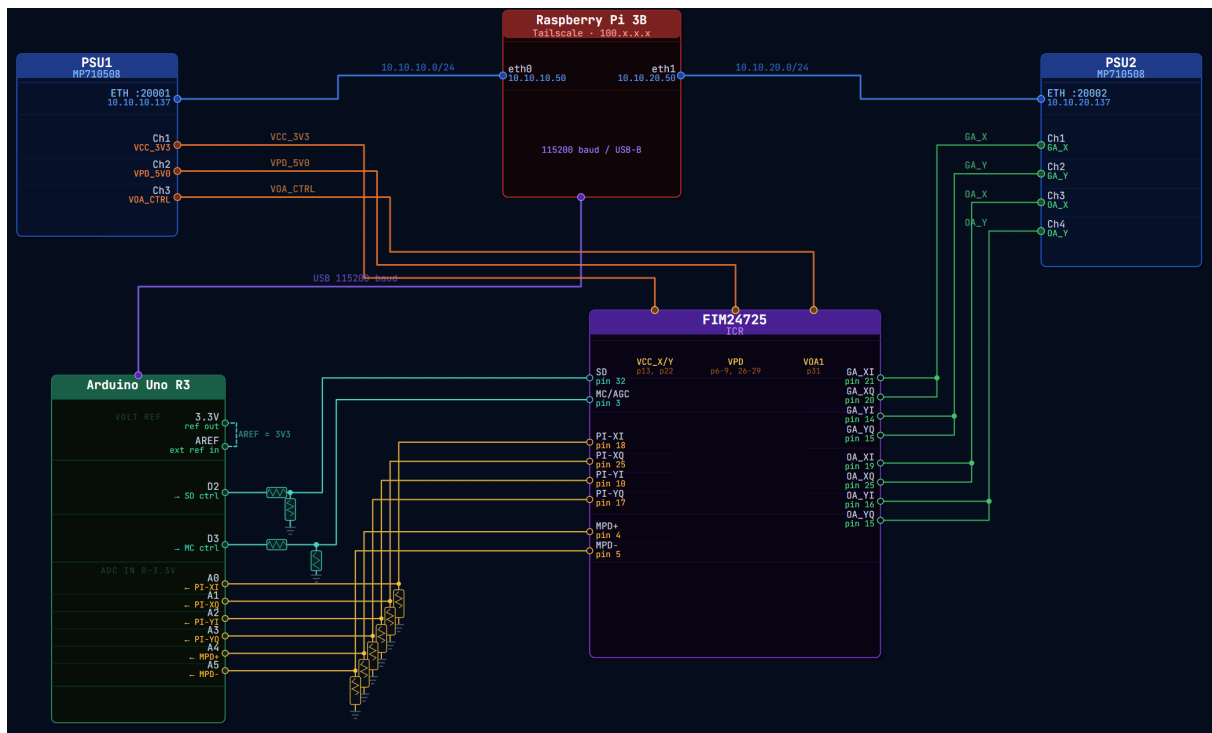
The Pi 3B is not a powerful machine, it has a quad-core 1.2 GHz ARM Cortex-A53 processor and just 1 GB of RAM [4]. However, the workload is light: the system issues and receives a handful of UDP datagrams per second, and serves a web interface to at most a couple concurrent users. At no point did the Pi's compute or memory present a bottleneck.

Raspberry Pi OS (64-bit), which is a lightweight port of Debian Trixie provides native arm64 packages and a desktop environment. Pi OS was chosen because it provides tested hardware support for the Pi out of the box (Wi-Fi and peripheral drivers) and offers the standard Debian ecosystem including Docker and Tailscale (Discussed in Section 3.3).

The Pi's built in Ethernet port allows it to the PSU's over a dedicated subnet. It's Wi-Fi chip connects it to the laboratory network (or any network) and supports web access, and USB connection to the MCU provides serial communication between the two.

3.2.4 System Topology

As in Figure 3.5, the two MP710508 power supplies are each configured with a static IP on separate subnets connected over Ethernet (blue wires). Interestingly, the MP710508 requires port of the sender be the same as it's own port, so sender ports must also be statically assigned and cannot be



randomly chosen by the OS. PSU1 resides on the 10.10.10.0/24 subnet at 10.10.10.137:20001, and PSU2 on the 10.10.20.0/24 subnet at 10.10.20.137:20002. The Arduino connects to the Pi via USB (purple wire) and is assigned device path (`/dev/arduino`) through a udev rule matching the FTDI FT232R's (USB chip) identifier. This ensures that the serial device name does not change if other USB devices are connected or the enumeration order varies between reboots. The rest of the connections are copper wires, with pull-down and divider resistors also shown.

3.3 Software Design and Implementation

This section presents the software architecture of the control system in a bottom-up approach: from hardware communication to the user interface. Python was chosen as the primary implementation language for this project because of my familiarity with it, and it's well-documented ecosystem for socket communication, server frameworks, and API and GUI frameworks which are needed here. Python's dynamic typing and interpreted execution makes it slower than compiled alternatives like C++ (used in the MCU firmware), but this is not a meaningful constraint for this application, where communication delays dominate over execution delays.

3.3.1 Hardware Abstraction Layer (HAL)

As noted in section 3.2.1, the MP710508 doesn't support multiple outstanding commands or tagged responses. The MP710508 communicates over used datagram protocol (UDP) using a proprietary ASCII protocol (see Appendix A1.1). Each command is a short string such as `VSET1:3.300`, which would set the voltage on channel 1 to 3.3V. The PSU replies with a datagram containing the result. There is no session, no handshake, no built-in sequencing, this must be driven by the caller. By design, the HAL is a thin policy-free (NFR2.1) translation layer that maps Python method calls to the MP710508's UDP/ASCII protocol and returns typed results.

The HAL translates these raw commands into Python methods. As in Figure 3.6, the HAL models the physical hardware directly with three classes:

- **PsuTransportUDP**: encapsulates a single UDP socket bound to one PSU's IP address and port. This is passed to the main MP71050x class, which means the communication method can be substituted (i.e. to USB or something else). It manages byte-level encoding, timeout management, and receiving replies.
- **MP71050x**: represents one physical PSU. It holds a reference to it's transport and provides device-wide operations of identifying the instrument, reading the status, enabling channels, setting values, etc. It contains a `threading.Lock`, which forces commands to execute in the order they are received, effectively serialising commands on a per-PSU basis. Because UDP is connectionless and the PSU processes one command at a time, interleaving commands from concurrent threads could corrupt a query-response pair. The lock ensures that a `set_voltage` on one thread cannot interleave with a `measure_voltage` on another.
- **Channel**: represents one output channel (CH1-CH4) on a specific PSU. `psu.channel(n)` binds the channel to its parent PSU. It's a self-contained object that can set voltage and current, read back setpoints, measure live output values, and configure over-voltage and over-current protection.

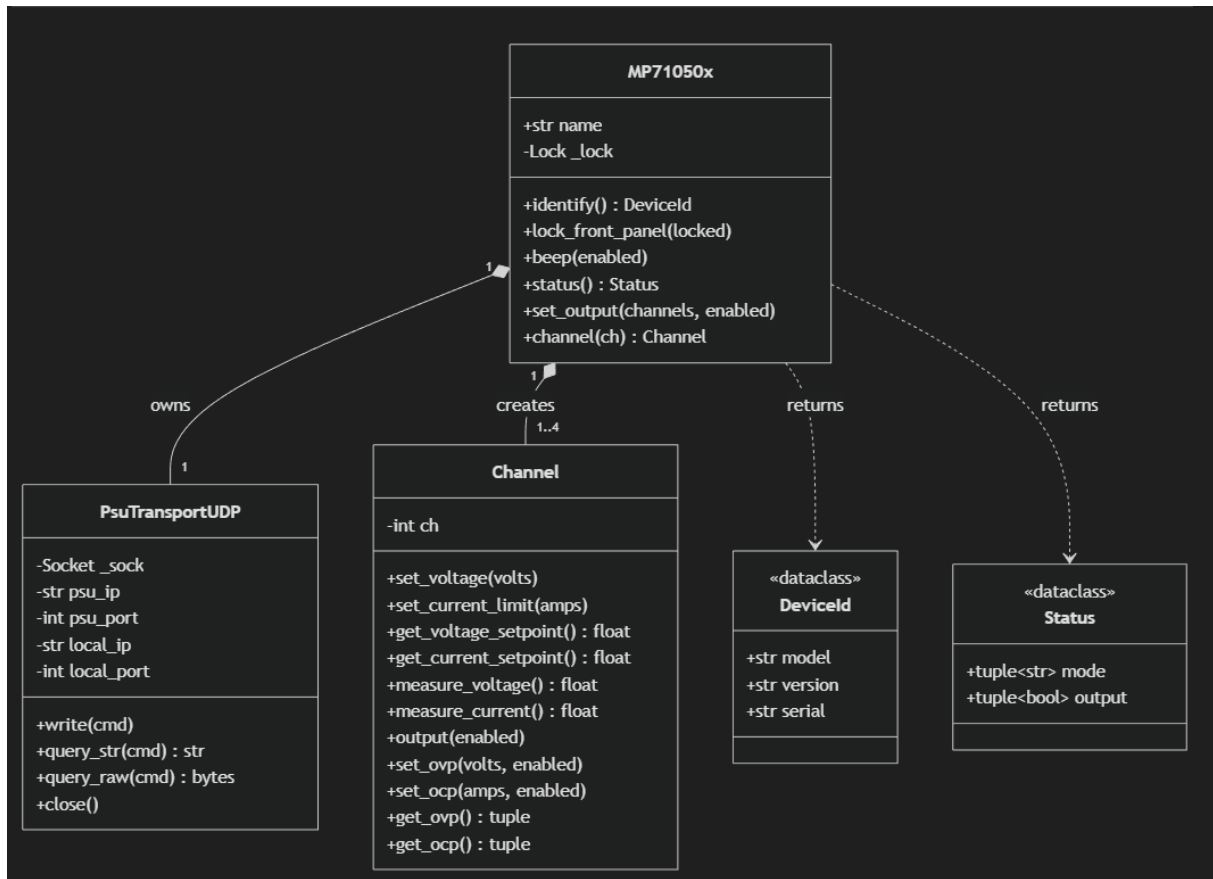


Figure 3.6: PSU HAL UML Class Diagram

Example usage:

```

from PSU_HAL import PsuTransportUDP, MP71050x

psu_a = MP71050x(
    "PSU_A",
    PsuTransportUDP(
        psu_ip="10.10.10.137", psu_port=20001,
        local_ip="10.10.10.50", local_port=20001
    )
)

ch1 = psu_a.channel(1)

ch1.set_voltage(5.0)
ch1.set_current_limit(0.5)
ch1.output(True)

print(ch1.measure_voltage(), ch1.measure_current())

ch1.output(False)
  
```

As mentioned in Section 3.2.4, each PSU is programmed with a static IP address and port. PSU1

resides on the 10.10.10.0/24 subnet at 10.10.10.137:20001, and PSU2 on the 10.10.20.0/24 subnet at 10.10.20.137:2000. The transport persistently binds to the same port number as the target PSU, which is required by the MP710508. The transport layer is deliberately isolated, if the communication method were to change only the transport class would need to be replaced (NFR2.2). All HAL calls are synchronous and blocking (`threading.Lock`) so that commands are executed sequentially (NFR1.3) with at most one command in-flight at a time. This directly reflects the PSU's protocol constraint and eliminates any risk of command interleaving or reply mismatch. The HAL exposes one method per conceptual operation, rather than raw strings. `send("VSET1:3.300")` becomes `channel.set_voltage(3.3)`. All commands sent and responses received are logged to a dedicated file (`logs/psu_hal.log`) with timestamps, PSU IP address, direction (TX/RX), channel, and command content (FR3.2). This provides a complete audit trail. Details of logging are discussed further in Section 3.3.6.

Arduino HAL

The Arduino (MCU) communication is also part of the HAL, providing digital control of SD, and MC/AGC, and producing six analogue monitor outputs. A custom ASCII based serial communication was designed for the Arduino. Commands are newline terminated strings, with responses echoing the applied state, similar to the PSUs. There is no binary framing, no packet structure, and no checksumming as simplicity was favoured given the expected robustness of a short USB link.

Table 3.13: MCU serial protocol.

Command	Response	Effect
HELLO	INIT:SD=1,MODE=AGC	Handshake. Re-asserts safe state (module disabled, AGC mode). Enables telemetry streaming.
PING	PONG	Liveness check. No side effects.
SD:1	SD:1	Drive D2 LOW : module disabled (shutdown active).
SD:0	SD:0	Drive D2 HIGH : module enabled (shutdown inactive).
MODE:AGC	MODE:AGC	Drive D3 HIGH : automatic gain control.
MODE:MGC	MODE:MGC	Drive D3 LOW : manual gain control.
SD:*	ERR:PIN_FAULT:SD	Pin readback mismatch after write.
MODE:*	ERR:PIN_FAULT:MODE	Pin readback mismatch after write.
(unrecognised)	ERR:UNKNOWN:<cmd>	Command string not recognised.
(unsolicited)	TELE:PI_XI=x.xxx,...,MPD_N=x.xxx	Periodic ADC telemetry (6 channels, every 500 ms). Enabled after HELLO.

The arduino boots automatically when it has power, and waits for the initialisation handshake (NFR2.3). If the handshake fails, an error is raised on the host side (FR1.6, NFR1.1) which puts the system into FAULT. A PING/PONG liveness check is also available for explicit health verification during startup.

Every command follows a command-response pattern that allows the service layer to ensure that

commands have actually been executed (NFR1.2). If a write command fails, that is detected and reported, with an error raised on the host side. The system state should only be maintained at the service layer (all persistent state lives in the service layer on the Pi, NFR2.1), so the Arduino maintains no system state information, every command is executed and verified with a digital read. Pin readback verification is performed after every `digitalWrite()`: the firmware immediately reads the pin back with `digitalRead()` and, if a mismatch is detected, emits an `ERR:PIN_FAULT:*` message instead of the normal echo. This provides a basic hardware integrity check.

Once the connection is initialised, the Arduino autonomously pushes telemetry every 500 ms (FR3.1). This was deemed to be sufficient time resolution for PI and MPD data, which is fairly static, does not change quickly, and serve more as operational indicators.

On the Pi, the `ArduinoMCU` class implements the `MCUInterface` protocol. A separate daemon thread continuously consumes serial data to read telemetry data and stores it in a cache. All other lines (command responses, errors) are placed on a queue for the synchronous command path. This separation means telemetry acquisition never blocks a control command, and control commands never cause a telemetry gap (NFR1.4). When the service layer or API needs a peak indicator reading, it reads from the cache.

The service layer does not depend on `ArduinoMCU` directly. Instead, it accepts any object satisfying the `MCUInterface` protocol which is a Python structural type defining `set_shutdown()`, `set_mode()`, `read_peak_indicators()`, `read_mpd()`, and `is_connected()`. This means that if another MCU were used, no service layer code would need to change. A `MockMCU` class implements the same `MCUInterface` protocol with randomised telemetry values and immediate command acknowledgement. This allows testing without any physical MCU connected, which was useful for development. The choice between real and mock MCU is made via an environment variable at application startup (NFR2.6).

3.3.2 Service Layer: `FIM24725Service`

The service layer is the brains of the operation, and safety boundary of the control system. It is where all the 'business logic' and safety-critical decisions: sequencing, state management, parameter validation are made and enforced (NFR2.1). Every path to hardware, whether initiated through a GUI button press, an API call, or future automation script, passes through the service. It follows a design philosophy of composition over inheritance, as shown in Figure 3.7, `FIM24725Service` owns a `StateMachine`, a `RailController`, and an `MCUInterface`, which are constructed or injected at initialisation, not subclassed; each component has a single responsibility.

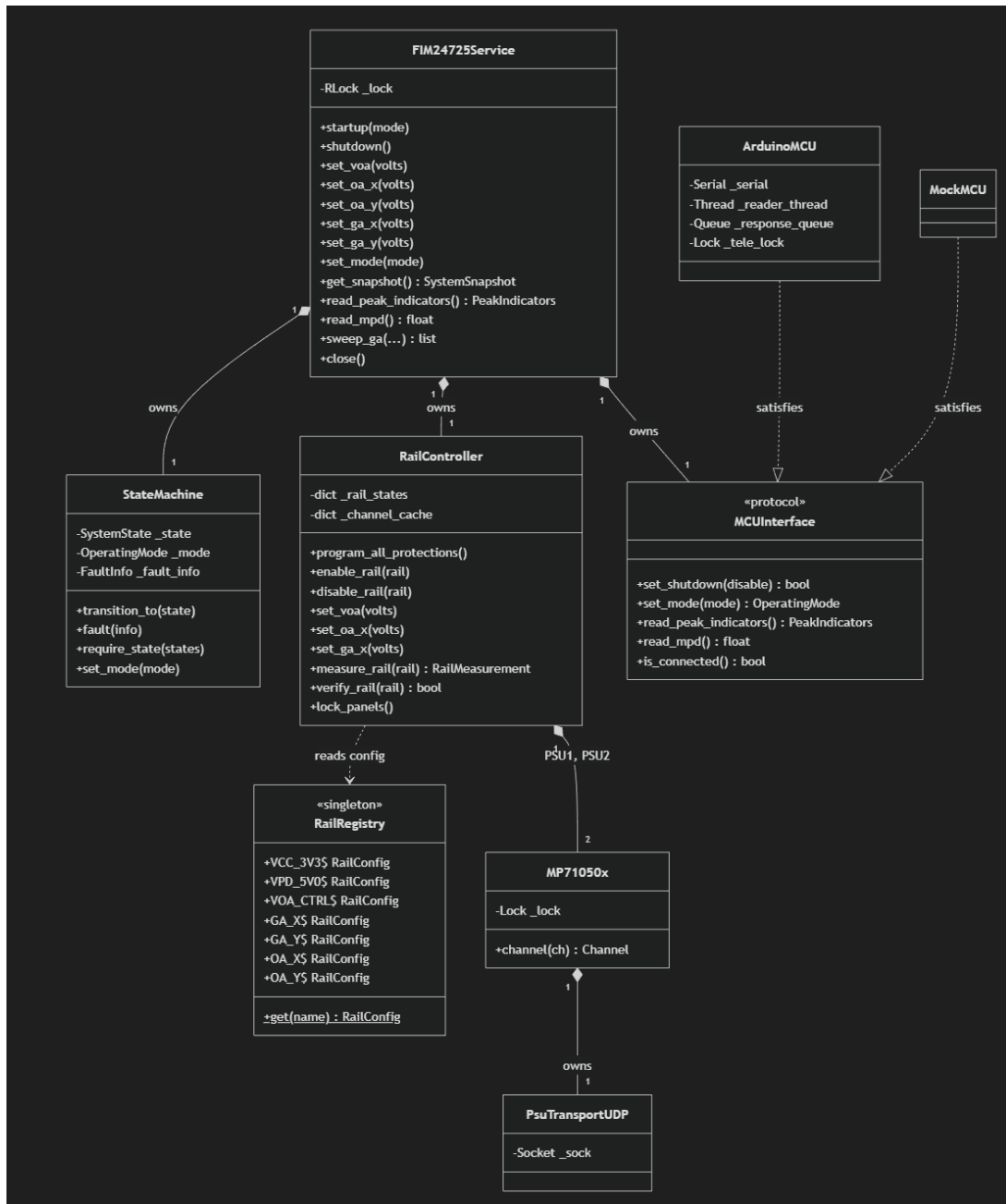


Figure 3.7: Service Layer UML Class Diagram

The problem it addresses

The service must orchestrate two PSUs (seven channels), and a MCU to manage the receiver's lifecycle. It must implement startup and shutdown sequences, validate all voltage setpoints against safe bounds (FR2.2), manage the systems state (FR1.5), handle faults by attempting a best-effort emergency shutdown (FR1.7), and do all of this safely under possible concurrent access from multiple API clients (NFR1.3). The HAL has provided the communication primitives but enforces none of these policies.

How rails are represented

The service defines the channel mapping described in Tables 3.10 and 3.11 in a RailRegistry, mapping human-readable names to their physical PSU channel, and defining their properties.

Example excerpt:

```
VPD_5V0: ClassVar[RailConfig] = RailConfig(  
  name=RailName.VPD_5V0,  
  psu_name="PSU1",  
  channel=2,  
  nominal_voltage=5.000,  
  nominal_current=0.050, # pd reverse bias current microA-mA  
  max_voltage=5.000,  
  ovp=5.500,  
  ocp=0.150,  
  verify_voltage=(4.75, 5.25),  
)
```

This registry is defined once and is the source of truth for all rail configuration. No other part of the codebase contains PSU channel numbers or voltage limits. A RailController class consumes the registry and provides named operations: `enable_rail("VPD_5V0")`, `set_voltage("VOA_CTRL", 2.5)`. Internally, it resolves the PSU and channel from the rail name based on the RailRegistry. This is where bounds are enforced, a command is checked against the RailRegistry, if it is outside the bounds, it is automatically clamped within safe bounds and a warning is logged (FR2.2). Actual hardware command is delegated to the HAL. After each write, the RailController waits 400 ms of settling time and reads back the actual PSU output and compares it against a tolerance window. A mismatch raises a `VerificationError`, which the service layer catches and converts to a fault (NFR1.2). Each rail is independently tracked as OFF, ENABLED, or FAULT. This per-rail state is returned alongside live voltage and current measurements in the system snapshot.

State Machine (FR1.5)

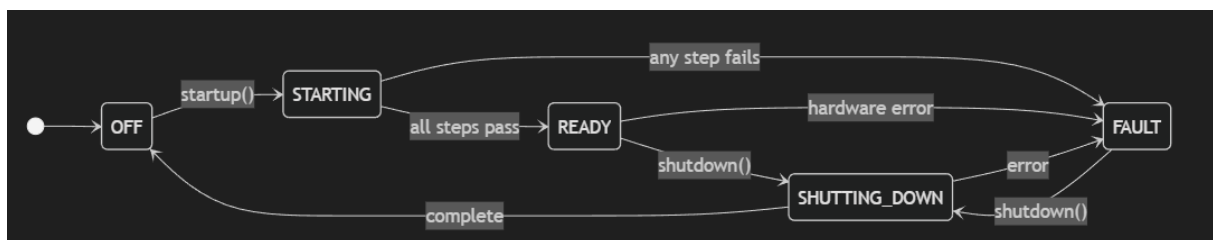


Figure 3.8: Control System State Machine Diagram

Figure 3.8 shows how the receiver's lifecycle is modelled in the StateMachine module.

- **OFF**: the initial state. Receiver not energised. Only `startup()` is permitted.
- **STARTING**: the startup sequence is in progress. No external operations are permitted.
- **READY**: all hardware is enabled and verified. Runtime operations permitted.

- **SHUTTING_DOWN**: the shutdown sequence is in progress. No external operations are permitted.
- **FAULT**: an error has occurred. Only shutdown() is permitted, to transition back to OFF.

Every public method implemented in the service checks the state before executing. Valid transitions are defined declaratively in a TRANSITIONS dictionary, not scattered through if/else chains:

```
TRANSITIONS: dict[SystemState, set[SystemState]] = {
    SystemState.OFF:{SystemState.STARTING, SystemState.FAULT},
    SystemState.STARTING:{SystemState.READY, SystemState.FAULT},
    SystemState.READY:{SystemState.SHUTTING_DOWN, SystemState.FAULT},
    SystemState.FAULT:{SystemState.SHUTTING_DOWN},
    SystemState.SHUTTING_DOWN:{SystemState.OFF, SystemState.FAULT},
}
```

If the state does not permit the requested transition, a `StateError` is raised and the request is rejected. This is how FR1.5 is implemented: a voltage-set command requested when the system is OFF is structurally impossible to execute. The FAULT state is reachable from and other state (FR1.7). A callback mechanism notifies FIM24725Service on every state change. The service registers a callback that triggers `_emergency_shutdown_unlocked()` whenever the state becomes FAULT during STARTING, allowing a best effort shutdown to take place. Other faults must be manually cleared to transition back to SHUTTING_DOWN and then OFF. Gain control mode (MC/AGC) is tracked as an orthogonal state within READY and not as a separate state in the main machine. State transitions and fault information are logged with timestamps (FR1.6).

Power Sequences (FR1.1)

The Startup and Shutdown sequences defined in Algorithms 1, 2 below implement the complete power-up and power-down procedure satisfying FR1.1. Startup must be explicitly called and does not run on startup (FR1.2). Fault states must be manually cleared by initiating a shutdown, except in the case that there is a fault during the startup sequence where a best-effort shut-down is initiated without manual intervention. The rationale behind this is that a startup fault can safely be automatically cleared as there should be no input signal applied, which is not the case for active READY or shutting down states. The Algorithm 1 implements FR1.3, where OVP and OCP are programmed prior to output being enabled and FR1.4 where PSU panels are locked as a first step.

Algorithm 1: FIM24725 Startup Sequence

Input: Initial operating mode $m \in \{AGC, MGC\}$

Precondition: System state is OFF

Postcondition: System state is READY with mode m , or FAULT

Transition state: OFF $\rightarrow$ STARTING;

/ Step 0: Lock PSU panels */*

foreach $PSU \in \{PSU1, PSU2\}$ **do**

└ Lock front panel, verify reachable;

/ Step 1: Establish initialisation baseline */*

Assert MCU responds to *PING*;

Set SD $\leftarrow$ DISABLE, verify echo;

Set MODE $\leftarrow$ AGC, verify echo;

Disable all PSU outputs are disabled;

/ Step 2: Program hardware protections */*

foreach rail $r \in \{VPD, VCC, VOA, GA_X, GA_Y, OA_X, OA_Y\}$ **do**

└ Program OVP(r), OCP(r), VSET(r), ISET(r) per RailRegistry;

└ Read back all four setpoints;

└ **if** any readback $\notin$ tolerance **then** raise VerificationError;

/ Step 3: Enable photodiode bias */*

Enable VPD_5V0, wait 500 ms;

$v_{meas} \leftarrow$ measure VPD voltage;

if $v_{meas} \notin [4.75, 5.25]$ **then** raise VerificationError;

/ Step 4: Enable amplifier supply */*

Enable VCC_3V3, wait 500 ms;

$v_{meas} \leftarrow$ measure VCC voltage, $I_{meas} \leftarrow$ measure VCC current;

if $v_{meas} \notin [3.135, 3.465] \vee I_{meas} \notin [0.12, 0.48]$ **then** raise VerificationError;

/ Step 5: Set control voltages */*

foreach control rail $r \in \{VOA, GA_X, GA_Y, OA_X, OA_Y\}$ **do**

└ Enable r ;

└ **if** any readback $\notin$ tolerance **then** raise VerificationError;

/ Step 6: Settle */*

Wait 500 ms ;

/ Step 7: Enable module */*

Set MODE $\leftarrow m$; verify echo;

Set SD $\leftarrow$ ENABLED (module on), verify echo;

/ Step 8: Start reading PIs */*

pi $\leftarrow$ read peak indicators from MCU;

foreach $p \in \mathbf{pi}$ **do**

└ **if** $p < 0.1\text{ V}$ **or** $p > 1.9\text{ V}$ **then** log warning (PI railed);

Transition state: STARTING $\rightarrow$ READY;

/ Exception handling (applies to all steps above) */*

if any step raises an exception **then** Transition state $\rightarrow$ FAULT;

Algorithm 2: FIM24725 Shutdown Sequence

Precondition: System is READY or FAULT**Postcondition:** System state is OFF with all outputs disabledTransition state $\rightarrow$ SHUTTING_DOWN;

```
/* Step 1:  Disable module output                                */
try
  Set SD  $\leftarrow$  DISABLED ;

/* Step 2:  Zero all control voltages                            */
foreach rail  $r \in \{GA\_X, GA\_Y, OA\_X, OA\_Y, VOA\}$  do
  try
    Set  $r \leftarrow 0V$ ;
    Disable output on  $r$ ;

/* Step 3:  Disable amplifier supply                            */
try
  Set VCC_3V3  $\leftarrow 0V$ , disable output;
  Verify VCC output is off;

/* Step 4:  Disable photodiode bias                             */
try
  Set VPD_5V0  $\leftarrow 0V$ , disable output;
  Verify VPD output is off;

/* Step 5:  Release PSU front panels                            */
try
  Unlock front panel on PSU1 and PSU2;

Transition state: SHUTTING_DOWN  $\rightarrow$  OFF;
Clear fault information;
/* Exception handling (applies to all steps above)              */
if any step raises an exception then Log error, continue;
```

Exception Hierarchy

All service-layer exceptions inherit from a common FIM24725Error base class, branching into domain-specific subtypes:

Table 3.14: Service-layer exception hierarchy.

Exception	Meaning
StateError	Operation invalid for current lifecycle state
BoundsError	Voltage outside rail's permitted range
VerificationError	PSU readback did not match commanded setpoint
MCUError	MCU communication failure or pin fault

Each exception type carries contextual information which propagates up to the operator through the API's error response described in Section 3.3.3. The API layer maps exceptions to appropriate HTTP status codes but the service layer itself is transport-agnostic and raises Python exceptions.

Thread Safety (NFR1.3, NFR1.4)

All public methods on FIM24725Service are decorated with @synchronized, which acquires a threading.RLock for the duration of the call (NFR1.3). RLock is used in place of Lock here because methods call each other internally and a non-reentrant lock would create a deadlock/permanent block. Reentrancy allows these internal call chains to proceed. Properties (state, mode, is_ready, fault_info) are also lock-protected, ensuring that a caller reading system status never sees a partially-updated snapshot. Synchronized decorator:

```
def synchronized(method: Callable[... , T]) -> Callable[... , T]:
    """
    Decorator to synchronize method access with the service lock.

    Uses RLock to allow reentrant calls from the same thread.
    """

    @wraps(method)
    def wrapper(self: FIM24725Service , *args: object ,
        **kwargs: object) -> T:
        with self._lock:
            return method(self , *args , **kwargs)

    return wrapper
```

Long operations such as startup and shutdown hold the lock for their entire duration, ensuring no other operation can proceed until they are completed. The practical consequence of this is that status polling for the GUI will block during these operations, but this is not a concern for short-running sequences, and polling information is irrelevant in the case of long running sequences like startup and shutdown.

MCU Integration

The service interacts with the MCU through the MCUInterface protocol, not a concrete class. The choice between ArduinoMCU (real hardware) and MockMCU (simulated telemetry) is made at construction based on an environment variable (NFR2.6).

Fault propagation (NFR1.1): Any MCUError caught during a runtime operation is passed to _fault_on_mcu_error(), which transitions the state machine to FAULT. This is also the path for MCU disconnect detection: if the Arduino is physically disconnected or loses power, the background reader thread in ArduinoMCU detects the serial exception.

Telemetry access (FR3.1): PI and MPD readings are accessed via mcu.peak_indicators and mcu.mpd_voltage, which return the latest values from the background reader thread's cache. These are included in the system snapshot described below. Telemetry acquisition runs on the Arduino's 500 ms timer independently of the service's command path, satisfying NFR1.4: telemetry does not block or delay control operations.

System Snapshot

The `get_snapshot()` method assembles a snapshot of the system properties. The snapshot includes: the current lifecycle state, operating mode, all seven rail measurements (voltage, current, CV/CC mode, and per-rail state), peak indicator values, MPD voltage, and any active fault information (FR1.6). This allows the GUI to poll the service for system information at a single endpoint (FR3.1).

Example system snapshot:

```
SystemSnapshot(  
    state=SystemState.READY,  
    mode=OperatingMode.AGC,  
    rails={  
        RailName.VCC_3V3: RailMeasurement(  
            name=RailName.VCC_3V3,  
            voltage=3.301,  
            current=0.372,  
            state=RailState.ENABLED,  
            mode="CV",  
        ),  
        RailName.VPD_5V0: RailMeasurement(  
            name=RailName.VPD_5V0,  
            voltage=4.998,  
            current=0.003,  
            state=RailState.ENABLED,  
            mode="CV",  
        ),  
        RailName.VOA_CTRL: RailMeasurement(  
            name=RailName.VOA_CTRL,  
            voltage=2.500,  
            current=0.041,  
            state=RailState.ENABLED,  
            mode="CV",  
        ),  
        RailName.GA_X: RailMeasurement(  
            name=RailName.GA_X,  
            voltage=0.000,  
            current=0.000,  
            state=RailState.ENABLED,  
            mode="CV",  
        ),  
        ...  
    },  
    sd_enabled=True,  
    peak_indicators=PeakIndicators(  
        pi_xi=1.234,  
        pi_xq=0.876,  
        pi_yi=1.543,  
    ),  
)
```

```
        pi_yq=0.234,
    ),
    mpd_value=0.444,
    mpd_n_value=0.123,
    fault_message=None,
)
```

3.3.3 API Layer

The web-based GUI and any other future automation clients need a well-defined, network accessible interface to access the service. This is the role that the application programming interface (API) layer fills. The API exposes all service functionality over HTTP without introducing any additional service logic.

FastAPI was selected as the web framework for it's ease of integration with Python-based tooling. FastAPI is an asynchronous server gateway interface (ASGI) framework that builds backend service gateways efficiently [23]. Its asynchronous execution model enables the system to handle multiple concurrent client requests. It integrates tightly with Pydantic for request body validation, allowing input constraints to be declared as type fields on request models and enforced automatically. Additionally, it also automatically generates documentation, helping with development and testing [23].

Concurrency Model

All of the APIs route handlers are defined as synchronous functions, not asynchronous. FastAPI automatically dispatches synchronous handlers into it's internal thread pool, where they are executed on worker threads [23]. Because the service layer uses `threading.RLock` for serialisation, an asynchronous route handler would deadlock the server. As such it is necessary to use synchronous event handlers.

With synchronous handlers in the thread pool, concurrent HTTP requests naturally serialise. The API layer introduces no additional locking, queuing, or synchronisation of its own (NFR1.3). The consequence is that long-running operations such as startup will cause other requests to queue at the service lock, not fail, and complete once the operation finishes. This is acceptable for a typically single-user piece of laboratory equipment where actions are initiated on human timescales.

Application Lifecycle

The broader 'backend' application is defined at this layer using a factory pattern. A function constructs the FastAPI application, registering middleware and routes: `FIM24725Service` and `HAL` services are created inside a FastAPI context manager [23]. This is important for NFR2.3: The system shall survive host power cycles and restart automatically without manual intervention, because it ensures hardware layer ports and serial connections are released properly on shutdown. It ensures that those services don't get instantiated as globals and linger as stale or duplicate hardware connections.

The device starts in OFF state when the server launches, and the operator must explicitly initiate startup (FR1.2).

Endpoints

The API is versioned and organised into four logical groups: service controls, voltage controls, telemetry, and logs. The GET /status endpoint is the primary data source for the GUI. It returns a complete system snapshot in a single response.

FIM24725 Coherent Receiver API 1.0.0 OAS 3.1

/openapi.json

REST API for controlling the FIM24725 coherent receiver module. Exposes PSU rail control, MCU digital signals, and telemetry.

service ^	
GET	/api/v1/status Get Status
POST	/api/v1/startup Post Startup
POST	/api/v1/shutdown Post Shutdown
POST	/api/v1/mode Post Mode
controls ^	
POST	/api/v1/controls/voa Post Voa
POST	/api/v1/controls/oa_x Post Oa X
POST	/api/v1/controls/oa_y Post Oa Y
POST	/api/v1/controls/ga_x Post Ga X
POST	/api/v1/controls/ga_y Post Ga Y
POST	/api/v1/controls/sweep_ga Post Sweep Ga
telemetry ^	
GET	/api/v1/telemetry/peak_indicators Get Peak Indicators
GET	/api/v1/telemetry/mpd Get Mpd
logs ^	
GET	/api/v1/logs Get Logs

Figure 3.9: FIM24725 Coherent Receiver API Endpoints

Input Validation

Request bodies for voltage-control endpoints use Pydantic models with `Field` constraints that block requests that fall outside the operating ranges:

```
class VoaRequest(BaseModel):
    """VOA voltage setpoint - clamped to 0-4.8 V at the
    service layer."""
    voltage: Annotated[float, Field(ge=0.0, le=4.8)]
```

Exception Mapping

Table 3.15: Service layer exception to HTTP status code mapping.

Exception	HTTP Status	Meaning
StateError	409 Conflict	Operation not valid in the current system state.
BoundsError	422 Unprocessable Entity	Requested value outside the permitted range for the target rail.
VerificationError	502 Bad Gateway	PSU readback did not match the commanded setpoint (hardware fault).
MCUError	502 Bad Gateway	MCU communication failure (timeout, serial error, or pin fault).

In Table 3.15, service layer exceptions are mapped to HTTP status codes to allow the GUI to display messages with context. 409 indicates that the request is valid in principle but conflicts with the current system state, the client can retry after state changes (e.g. a voltage setpoint attempt during startup). 422 indicates that the request is properly formed but the value is invalid. 502 indicates the API server is functioning but an upstream dependency (PSU or MCU) failed, distinguishing between software and hardware bugs. It's worth noting that a `BoundsError` should never be raised due to the Pydantic field validation handling malformed request bodies, and the service layer is configured to clamp out of bounds values rather than raise a `BoundsError`, but the exception exists in the case that clamping behaviour was changed in configuration.

API Middleware

Control Endpoints that trigger hardware I/O are rate-limited to prevent the system from excessive requests on a per IP address using `SlowAPI`. Rate limiting is a standard API design practice used to protect system resources, ensure fair usage across clients.

The cross-origin resource sharing (CORS) scheme is configured to only allow requests that come from the localhost (from the same machine) which prevents external web-pages from interacting with the API. `TrustedHostMiddleware` complements this and is configured to only accept requests from localhost. API traffic never leaves the Pi, since the GUI is hosted there too. External access to the GUI is controlled at the network level by Tailscale (NFR3.1, NFR3.2), not by application-layer authentication. These fit into the security model of the system which is discussed in more detail in Section 3.3.5.

Table 3.16: API rate limits per endpoint group.

Group	Endpoints	Limit	Rationale
Status	GET /status	300/min	Polled by the GUI every quickly for updates.
Telemetry	GET /peak_indicators GET /mpd	300/min	Allows sub-second polling for real-time ADC readback.
Controls	POST /startup POST /shutdown POST /mode POST /voa POST /oa_* POST /ga_* POST /sweep_ga	10/min	Hardware I/O operations that should not be issued rapidly.
Logs	GET /logs	60/min	Delta-polled by the GUI log panel at a moderate interval.

Logs

Logs generated by the service are appended to a ring buffer, with each log assigned a monotonically increasing number. The GET /logs endpoint exposes this buffer with support for delta polling: the client passes since_seq (the highest sequence number from its previous response), and the server returns only records with a higher sequence number. This buffer is not persistent and is lost on system restart, it only serves to provide the GUI with a live feed. This polling strategy was chosen to reduce redundant data transfer and improve the responsivity of the GUI (FR3.2) without introducing complexity of persistent streaming mechanisms. The logging system for the service is discussed in more detail in Section 3.3.6.

3.3.4 Graphical User Interface (GUI)

The user interface must provide control, monitoring, and log-viewing functions through a standard web browser with no client-side installation (FR4.1, NFR2.4). It must reflect the current system state and prevent the operator from issuing commands that are invalid for that state (FR4.2), making it safe for a researcher unfamiliar with the FIM24725's internal constraints to operate the receiver (NFR2.7). It must present real-time telemetry in a form that allows the operator to observe system behaviour during adjustment (FR3.1).

Dash was selected as the UI framework. It's Python native which avoids introducing a separate JavaScript build toolchain. It provides built-in graphing components used for the telemetry display, and a components library which contains a set of styled UI components (buttons, inputs, toggles).

The trade-off is that Dash is less flexible than a full frontend framework however the control panels interaction model is comfortably within its capabilities.

The GUI communicates with the API over HTTP, with no direct access to hardware, the service layer, or Python objects in the background: it's purely a presentation layer.

UI Journey

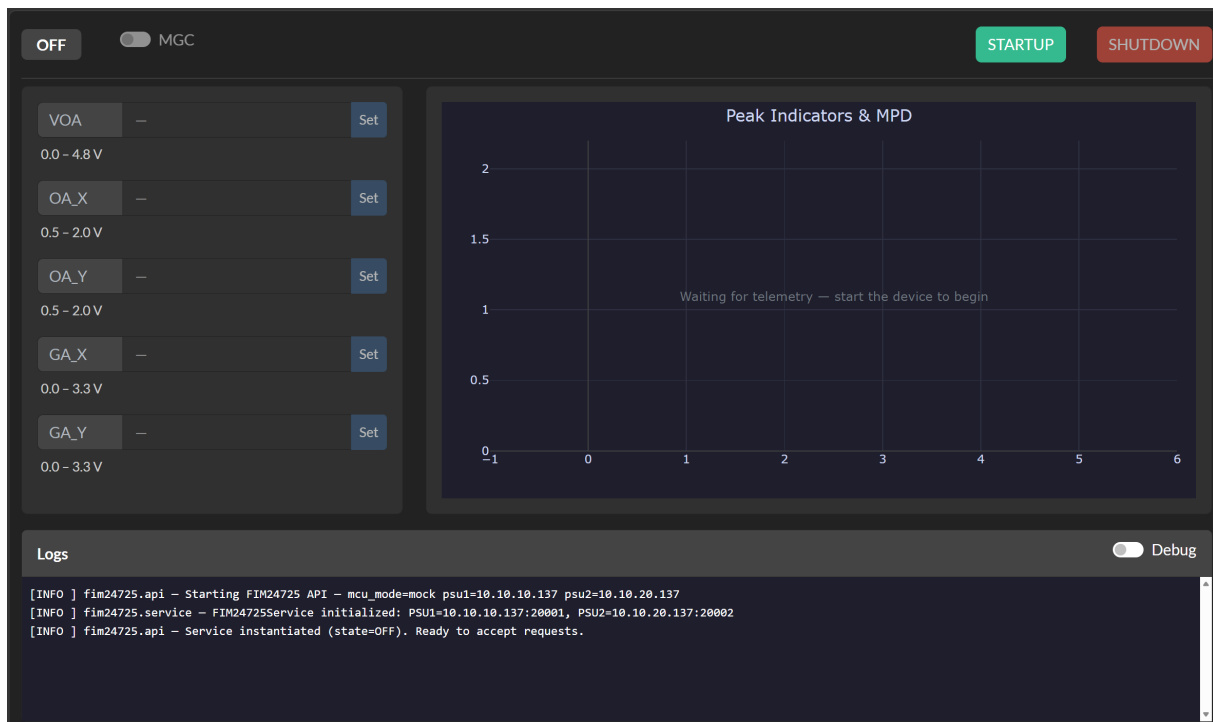


Figure 3.10: GUI — Landing page, system off.

Figure 3.10 shows the page that greets the user when connecting to the GUI. The interface is contained in a single page with all relevant information: state, control, telemetry, logs visible at once.

The **header bar** at the top of the page contains the most critical elements. The system state badge (OFF here) offers the most critical piece of information: “is the system on, and is anything wrong?” It is accompanied by actions that can affect the system state: a toggle slider for gain mode selection, and buttons for startup and shutdown.

The **controls panel** section on the left contains the controls for the adjustable rails. Each control has a numeric input field, with a set button, and the valid range displayed below. The set-buttons become bright and clickable once the system enters READY state. A control box will appear red if a user attempts to enter a value outside the operating range.

The **telemetry graph** panel on the right hand side displays peak indicator and monitor photodiode voltages over a 60 second rolling window. In the OFF state it displays 'waiting for telemetry'. The user can observe the effect of adjustments in near real-time, as telemetry is polled every 500ms. The user can also zoom and pan the graph to inspect a region of a trace. The graph is rendered client-side, keeping the page responsive, and was a deliberate choice to get around VPN latency (discussed in Section 3.3.5).

The **log panel** at the bottom of the screen displays diagnostic and audit messages, that assist when something unexpected happens. It has a 'debug mode' toggle button that alters the log panel to also display low-level debug messages such as PI readback values and hardware-level commands. The panel uses the delta-polling mechanism provided by the GET /logs endpoint (Section 3.3.3).

On initial load, the last 100 lines are fetched. Each subsequent poll (every 2 seconds) returns only newer entries. The log panel keeps a 500 line buffer and is independently scrollable, and scrolls automatically with incoming logs if the user is within 50 pixels of the bottom.

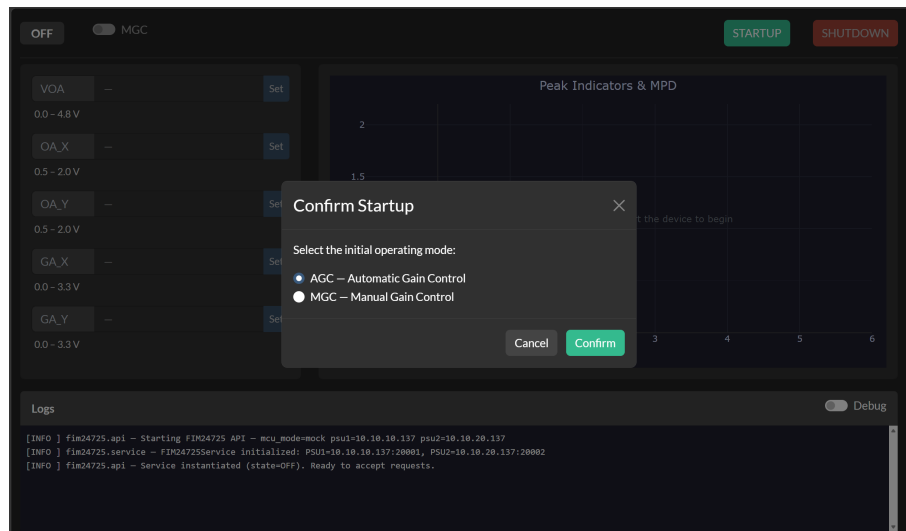


Figure 3.11: GUI — Startup confirmation with initial mode selection.

When clicking startup (Figure 3.11), the user is prompted to select the gain control mode they would like to start in, and confirm.

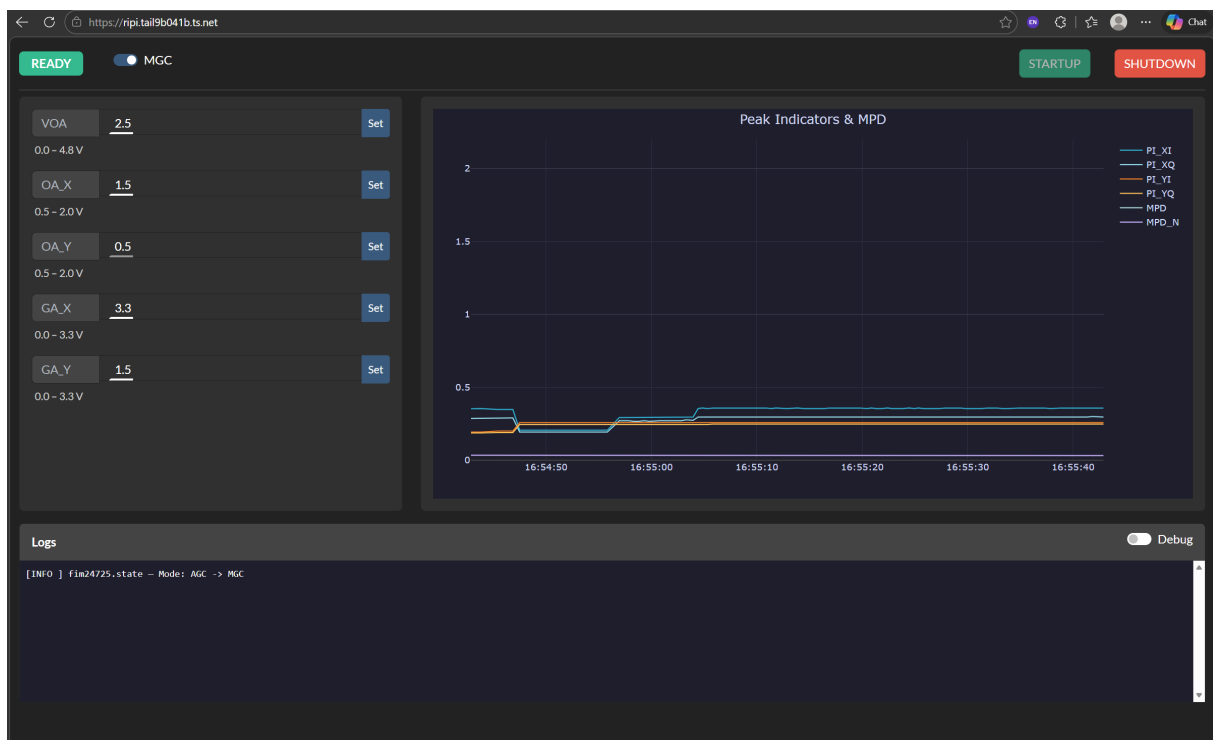


Figure 3.12: GUI — READY state MGC mode, with control values set and readback visible.

Figure 3.12 shows the GUI when the system is in the READY state. Here, control values can be set, indicators are graphed, and logs flow in. The state badge in the top left has transitioned to a green

READY, indicating system health, and control panel 'set' buttons have become available. The traces of the peak indicators and MPD can be seen in the graph panel. The blue traces represent the X polarisation of the receiver, the stepping-up behaviour seen here was achieved by increasing the gain on X (GA_X) by intervals. The mode transition from AGC to MGC can be seen in the log panel.

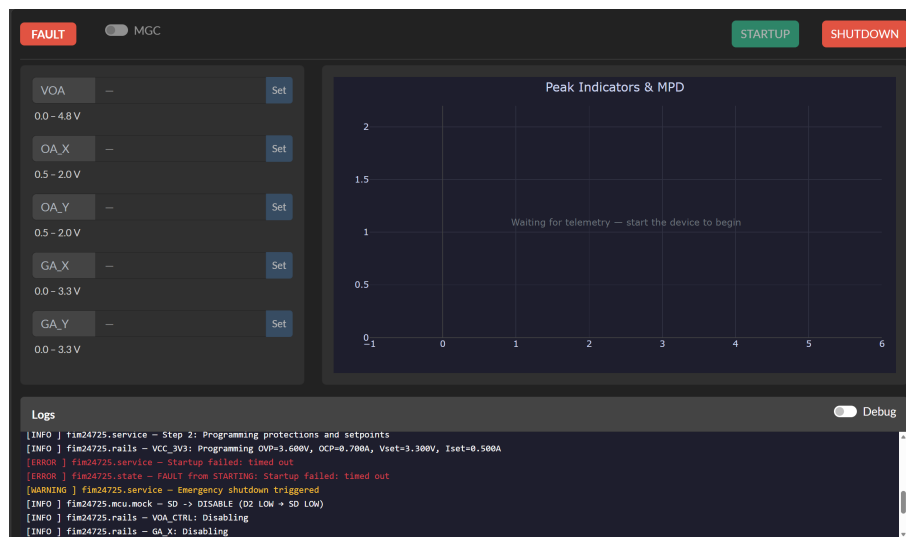


Figure 3.13: GUI — FAULT during startup sequence.

Figure 3.13 shows the GUI when a fault occurred during startup. The state badge has transitioned to a red FAULT. The logs display the fault message in red, in this case due to a timeout, as the PSUs were disconnected.

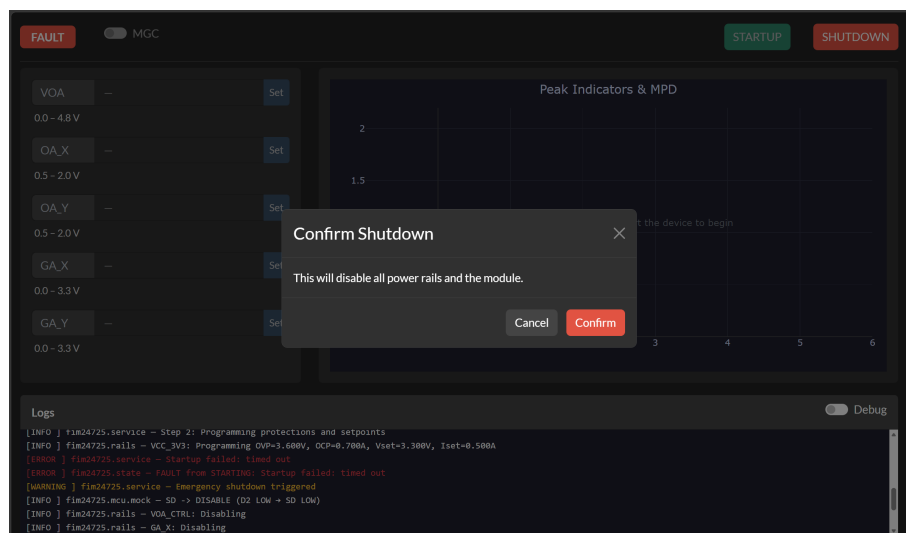


Figure 3.14: GUI — Shutdown confirmation.

In Figure 3.14, the user is prompted with the shutdown confirmation, ensuring it is an intended action.

3.3.5 Remote Access and Security

The control system is a single-user laboratory instrument deployed on a private network within the CONNECT laboratory. Exposing a service on the Raspberry Pi directly to the internet with an open port introduces significant security risks. An external host could discover and attempt to connect to the service making it directly accessible to automated scanning, probing, and exploitation attempts present on the public internet today. Without robust authentication, an attacker could issue commands, or more troublingly potentially exploit vulnerabilities in the API and compromise the Raspberry Pi [24]. As the Pi is connected to the lab network, such a compromise wouldn't be isolated, and could present opportunities for lateral movement, potentially accessing other systems within the network and increasing the overall attack surface.

To mitigate these risks, the system avoids exposing the API to the public internet entirely. Remote access is provided through Tailscale, a zero-configuration mesh VPN built on the WireGuard protocol [25], which is already used in the CONNECT lab for other systems, and was therefore a natural choice. It required no new infrastructure, and no firewall rule changes (NFR3.1, NFR3.2).

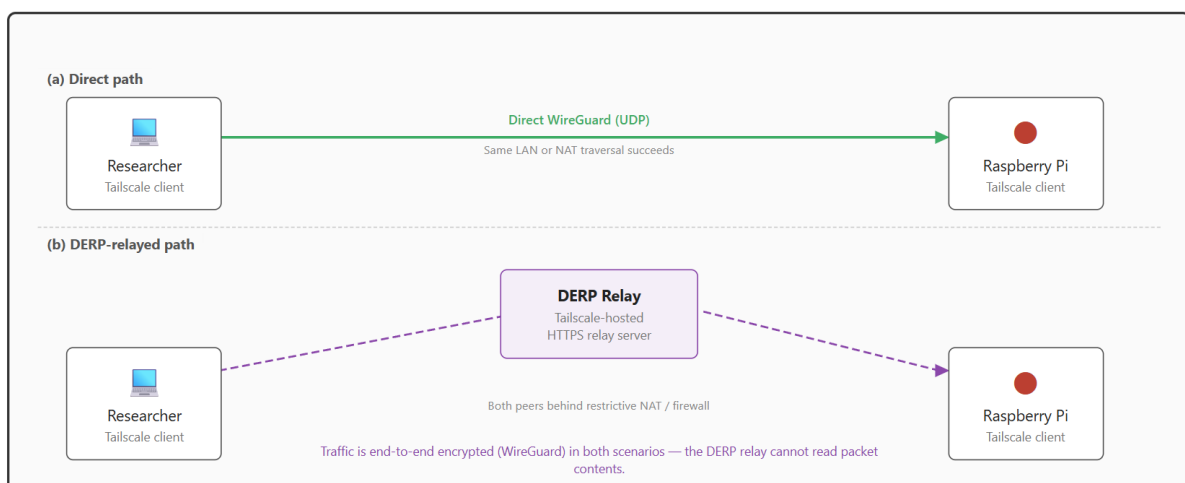


Figure 3.15: Tailscale Connection Paths (Direct and DERP Relay).

Tailscale operates by establishing encrypted point-to-point WireGuard tunnels between authenticated devices on a shared *tailnet* [25]. Each device is reachable only from other devices on the same tailnet. Traffic is end-to-end encrypted. For users on the lab network, a direct WireGuard tunnel will be established between the two devices. In the case that the user is not on the laboratory network, a direct tunnel cannot be established due to network address translation (NAT). In this case, Tailscale's DERP relay acts as an encrypted fallback forwarding proxy [25]. This allows connections to the Pi from the public internet for tailnet members. This makes the system accessible for any authorised user (who has been added to the lab tailnet by an administrator) from any network. The benefit of this is that it allows a user to check and control experiments without having to be on-premises, and opens up the ICR for use by authorised partner institutions.

An interesting consequence of a DERP relayed connections was that it added ~ 70 ms of latency on each round-trip request. The GUI polls the API every 500 ms for updates state and indicator

updates, with each poll opening a new TCP connection, and polls for log updates every 2000ms. Each TCP handshake for a new connection represents a delay of ~105 ms just to initiate the connection, and not including additional request-response time. These delays compounded, making the GUI sluggish and basically unusable for relayed connections. The fix was to replace per-request connections with a persistent `httpx.Client` session that reuses a single TCP connection, reducing subsequent requests to a single round trip and restoring the responsiveness of the interface.

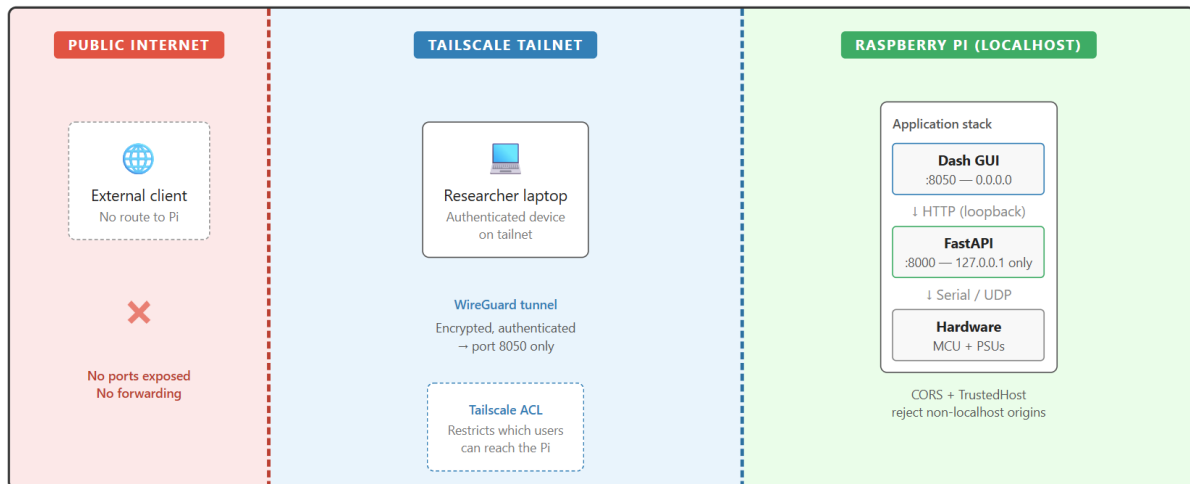


Figure 3.16: Network Access Zones for the Control System (Raspberry Pi).

Tailscale's ACL policy controls which devices can reach which ports. The policy configured for this deployment restricts inbound connections to the GUI port only:

```
{
  "acls": [
    {
      "action": "accept",
      "src": ["autogroup:member"],
      "dst": ["<pi-tailscale-ip>:8050"]
    }
  ]
}
```

This makes it so that authorised users (tailnet members) can only access the GUI and no other resources on the PI.

In addition to network-level control, the API enforces two-application level restrictions. Cross-Origin Resource Sharing (CORS) is restricted to `localhost`, which prevents browser-hosted scripts on other origins from issuing requests to the API. FastAPI's `TrustedHostMiddleware` complements this by rejecting `Host` headers that don't match `localhost`. The backend API binds to `localhost` only, so it is not directly reachable from the network, only the co-located Dash GUI can contact it. This effectively makes the GUI the only allowed origin, which is served from the same host and reaches the API via `http://localhost:8000`. The GUI binds to `0.0.0.0` because it must be reachable on the Pi's Tailscale interface (`tailscale0`) for remote access.

This means that API traffic never leaves the Pi, and the only traffic that crosses the network is the Dash GUI, which serves pre-rendered HTML and receives user interactions to/from the operator's browser.

These protections are lightweight, but appropriate for the isolated deployment model. The system does not implement application-level authentication or authorisation. For a single instrument, with a small team this is acceptable however it means that there are no user permission roles distinguishing between who can control versus who should only monitor. It also means, while there is an audit trail of actions, there is no trail of *whom* performed the action. A future iteration could aim to implement these by defining access roles, but it was not justified for the current deployment model.

3.3.6 Deployment and Configuration

The control system must recover unattended from power cuts due to moving the system around the laboratory, and must be movable to a different host (such as the lab servers) if circumstances required it. This section describes how the software is packaged, configured, and kept running.

Containerisation

The application is packaged as two separate Docker Containers: one for the FastAPI backend, and one for the Dash GUI. Separating the backend and GUI into distinct containers is in line with the separation of concerns ideals: a bug in the GUI will not take down backend control (NFR2.1). It also means that just the API could be run to interact with other future applications if desired.

The backend runs on Uvicorn, an ASGI (Asynchronous Server Gateway Interface) server. Uvicorn manages the event loop and dispatches synchronous route handlers into its thread pool as described in Section 3.3.3. The GUI runs on Gunicorn, a WSGI (Web Server Gateway Interface) server, because Dash is a Flask-based framework and Flask implements the WSGI standard [26].

Containerisation also provides platform portability (NFR2.5). The Python 3.12-slim base image has official variants for both arm64 and x86, so the same Dockerfiles produce working images on the Raspberry Pi, a developer's laptop, or a laboratory server without modification [27]. If the deployment target were to change, for example migrating to lab servers, one command would bring the system up on the new host, only environment variables pointing to the correct network and device interfaces would be required.

Automatic Restart

Both containers are configured with a Docker restart policy `restart: unless-stopped`. Combined with Docker's systemd service (which was enabled on the Pi at installation), this provides automatic recovery from power loss. When the Pi boots, systemd starts the Docker daemon which restarts both containers. The GUI container's `depends_on` directive with a healthcheck condition ensures it does not start until the API backend is ready to serve requests. No cron jobs, custom systemd units, or manual intervention are required (NFR2.3).

Configuration

All deployment-specific parameters are configured via environment variables declared in the environment block of `docker-compose.yml` (NFR2.6). Both the backend and GUI use Pydantic `BaseSettings` classes that load values from environment variables (prefixed `FUJ_` and `FUJ_GUI_` respectively), with coded defaults as fallbacks. This approach allows the same container images to serve different deployment scenarios without code changes:

Table 3.17: System deployment configuration parameters.

Variable	Deployed Value	Type	Description
<i>Backend (prefix: FUJ_)</i>			
PSU1_IP	10.10.10.137	str	PSU 1 IP address.
PSU1_PORT	20001	int	PSU 1 UDP port.
PSU1_LOCAL_IP	10.10.10.50	str	Local NIC IP bound for PSU 1.
PSU2_IP	10.10.20.137	str	PSU 2 IP address.
PSU2_PORT	20002	int	PSU 2 UDP port.
PSU2_LOCAL_IP	10.10.20.50	str	Local NIC IP bound for PSU 2.
MCU_MODE	real	str	real/mock for mocked MCU.
MCU_PORT	/dev/arduino	str	Serial device path of MCU.
API_HOST	127.0.0.1	str	Address the API server binds to.
API_PORT	8000	int	Port the API server listens on.
LOG_LEVEL	INFO	str	Log level for file and console output.
LOG_DIR	logs	str	Directory for rotating log files.
<i>GUI (prefix: FUJ_GUI_)</i>			
API_BASE_URL	http://localhost:8000	str	Backend API URL the GUI polls.
GUI_HOST	0.0.0.0	str	Address the Dash server binds to.
GUI_PORT	8050	int	Port the Dash server listens on.
DEBUG	false	bool	Enable Dash debug mode.
POLL_INTERVAL_MS	500	int	Status polling interval (ms).
LOG_POLL_INTERVAL_MS	2000	int	Log polling interval (ms).
GRAPH_HISTORY_S	60	int	Rolling time window for the telemetry graph (s).

Logging and Persistence

The backend writes to three rotating log files, mounted from the host via a Docker volume so they persist across container rebuilds:

Table 3.18: Log files, contents, and rotation policy.

File	Content
psu_hal.log	Raw PSU communication: every TX/RX
fim24725_service.log	Service-layer activity: state transitions, rail operations, faults
api.log	API request lifecycle and rate limit events

All three rotate daily at midnight with 30-day retention (FR3.2). The volume mount (./logs:/app/logs) ensures logs survive container restarts. All logs are timestamped, and provide an audit log of what has been commanded from the API level all the way down to the hardware level.

HAL logs record the timestamp, the PSU IP, transmission direction, and command:

```
2026-04-05 18:13:19.051 | 10.10.10.137 | TX | CH1 | OVPSET1:3.600
2026-04-05 18:13:19.082 | 10.10.10.137 | TX | CH1 | OVP1:1
2026-04-05 18:13:19.113 | 10.10.10.137 | TX | CH1 | OCPSET1:0.700
```

Service logs provide a timestamped, step-by-step audit trail of every hardware operation; rail programming, state transitions, MCU commands, the operator can trace exactly what the system did and in what order. Logs are colour-coded by level (white for INFO, amber for WARNING, red for ERROR, grey for DEBUG):

```
2026-04-05 18:31:23.400 | fim24725.service | INFO | Step 2: Programming protections
and setpoints
2026-04-05 18:31:23.400 | fim24725.rails | INFO | VCC_3V3: Programming OVP=3.600V, OCP
=0.700A, Vset=3.300V, Iset=0.500A
2026-04-05 18:31:25.586 | fim24725.service | ERROR | Startup failed: timed out
2026-04-05 18:31:25.587 | fim24725.state | ERROR | FAULT from STARTING: Startup failed
: timed out
2026-04-05 18:31:25.587 | fim24725.service | WARNING | Emergency shutdown triggered
2026-04-05 18:31:25.587 | fim24725.mcu.mock | INFO | SD -> DISABLE (D2 LOW -> SD LOW)
2026-04-05 18:31:25.588 | fim24725.rails | INFO | VOA_CTRL: Disabling
2026-04-05 18:31:25.618 | fim24725.rails | INFO | GA_X: Disabling
2026-04-05 18:31:25.649 | fim24725.rails | INFO | GA_Y: Disabling
2026-04-05 18:31:25.681 | fim24725.rails | INFO | OA_X: Disabling
2026-04-05 18:31:25.712 | fim24725.rails | INFO | OA_Y: Disabling
2026-04-05 18:31:25.743 | fim24725.rails | INFO | VCC_3V3: Disabling
2026-04-05 18:31:25.774 | fim24725.rails | INFO | VPD_5V0: Disabling
2026-04-05 18:31:35.738 | fim24725.service | INFO | === Starting FIM24725 Shutdown
Sequence ===
2026-04-05 18:31:35.739 | fim24725.service | INFO | Step 1: Disabling module output (
D2 LOW -> SD LOW = shutdown active)
2026-04-05 18:31:35.739 | fim24725.mcu.mock | INFO | SD -> DISABLE (D2 LOW -> SD LOW)
2026-04-05 18:31:35.739 | fim24725.service | INFO | Step 2: Disabling control rails (
VOA, GA, OA -> 0V)
```

The API log captures application lifecycle events: server startup, service instantiation confirmation, and clean shutdown. It's a separate audit trail from the service log, focused on the API process rather than hardware operations:

```
2026-02-25 23:08:54.381 | fim24725.api | INFO | Starting FIM24725 API - mcu_mode=mock
psu1=10.10.10.137 psu2=10.10.20.137
2026-02-25 23:08:54.382 | fim24725.api | INFO | Service instantiated (state=OFF).
Ready to accept requests.
2026-02-25 23:08:54.440 | fim24725.api | INFO | Shutting down - closing service and
MCU
```

3.4 Taking a Step Back: Overall System Architecture

The preceding sections described each layer of the control system, working up from hardware communication through the control system business logic, through to the user interface and security/access control. This section steps back and presents the complete system as an integrated whole: how the layers connect and how data flows through them.

From Figure 3.17, the entire user journey can be seen. The lab technician accesses the system through a browser via Tailscale's encrypted tunnel to the Raspberry Pi 3. This request reaches the GUI container (`fuj_gui`), running the Gunicorn server, which serves the technicians browser with the GUI. This GUI allows the technician to request operations (such as startup and shutdown), which are sent over persistent HTTP connections to the backend container (`fuj_api`) running the Uvicorn server. The FastAPI on this server parses the request details and routes them to the service layer (`fim24725_service`). This is where safety controls, validation, and orchestration of the MP710508 power supplies, and Arduino Uno MCU takes place. The service layer decides if the command is safe to execute by checking its rail registry and state machine. If it is, the service layer passes the command to the hardware abstraction layer (HAL), which in turn sends the necessary commands to the PSUs over UDP, and/or to the MCU over USB serial. These three devices (2 PSUs and MCU) are physically connected to the FIM24725 ICR. The PSUs receive the command from the HAL and execute it by, for example setting a voltage on a specific channel, or responding with information about a channel. In a similar manner, the MCU receives the command from the HAL, and executes it by setting digital shutdown or gain mode selection signals. Simultaneously, the MCU is reading telemetry from the FIM24725 via it's onboard ADC and sending that back to the backend, where it bubbles up through the layers and gets displayed on the lab technician's GUI.

The FIM24725 takes in two laser sources, a strong local oscillator and a signal laser, which is modulated with some information signal. The commands that propagate down to the FIM24725 as electrical control signals control the startup, shutdown, and gain adjustment settings, which enables the FIM24725 to perform coherent detection, and output the information signal (across its different polarisations and quadratures).

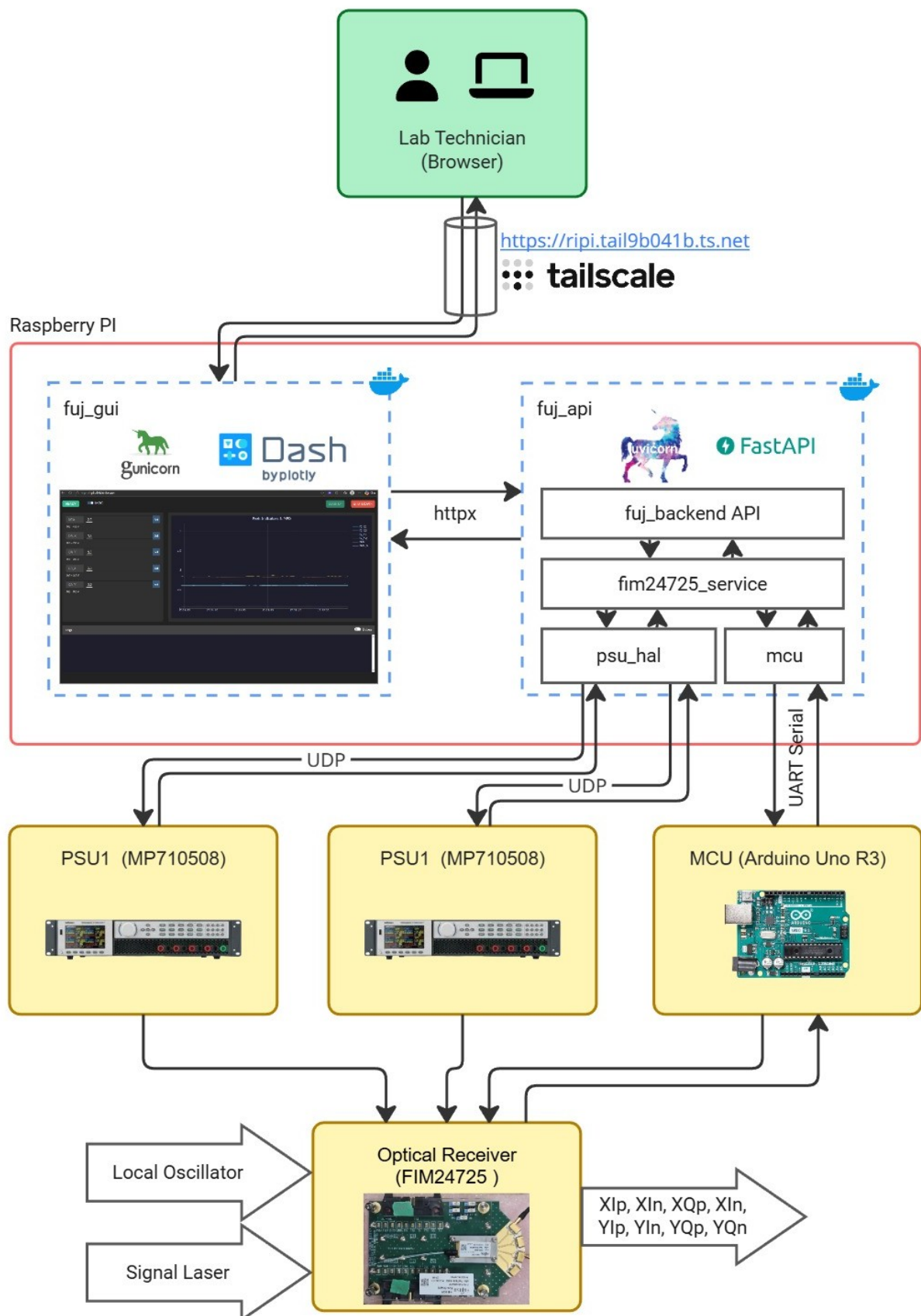


Figure 3.17: Comprehensive System Architecture Diagram.

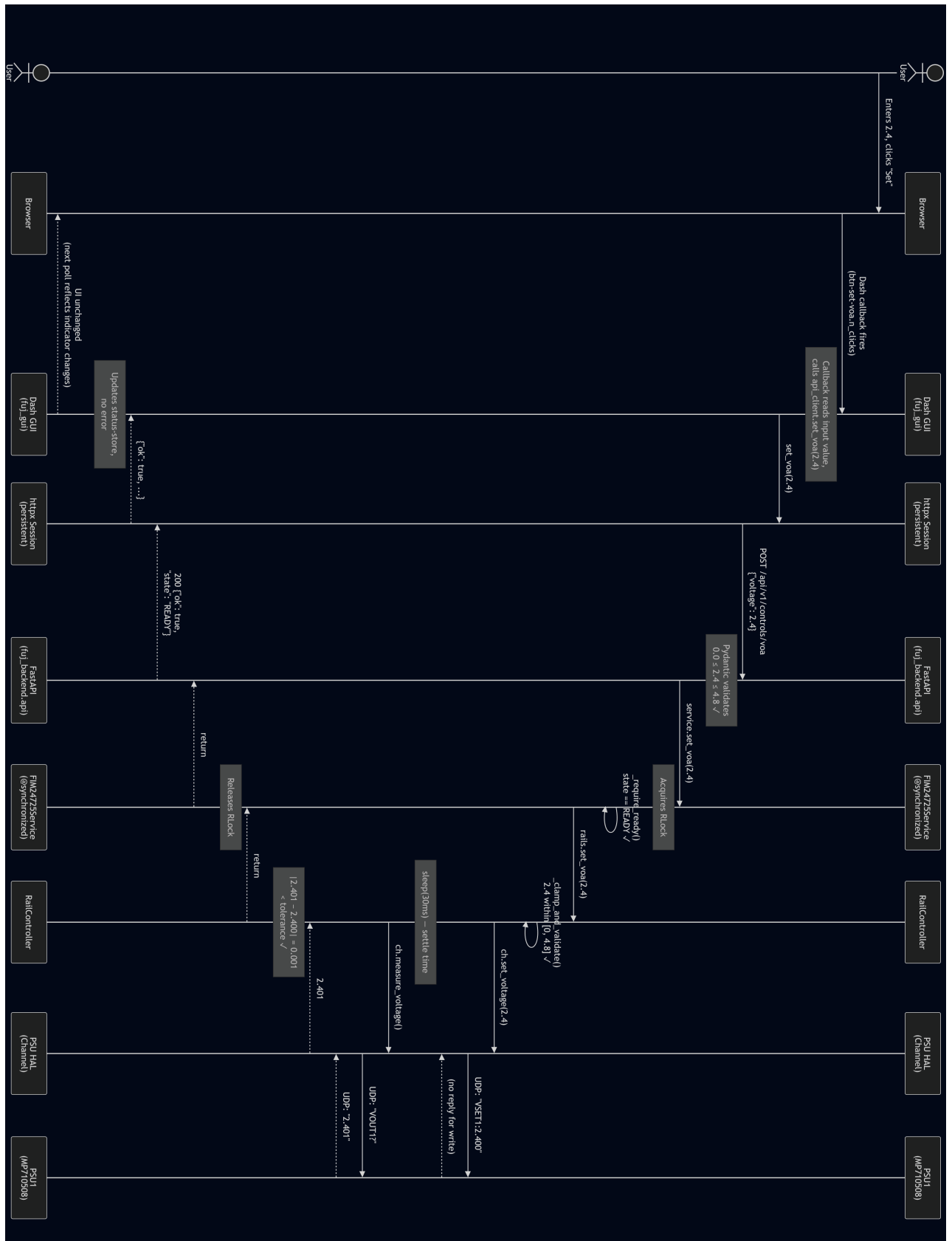


Figure 3.18: Example Data Flow Trace: Set VOA to 2.4V.

Figure 3.18 shows what the data flow looks like for a user command when the system is in the ready

state: setting the variable optical attenuation (VOA) to 2.4V.

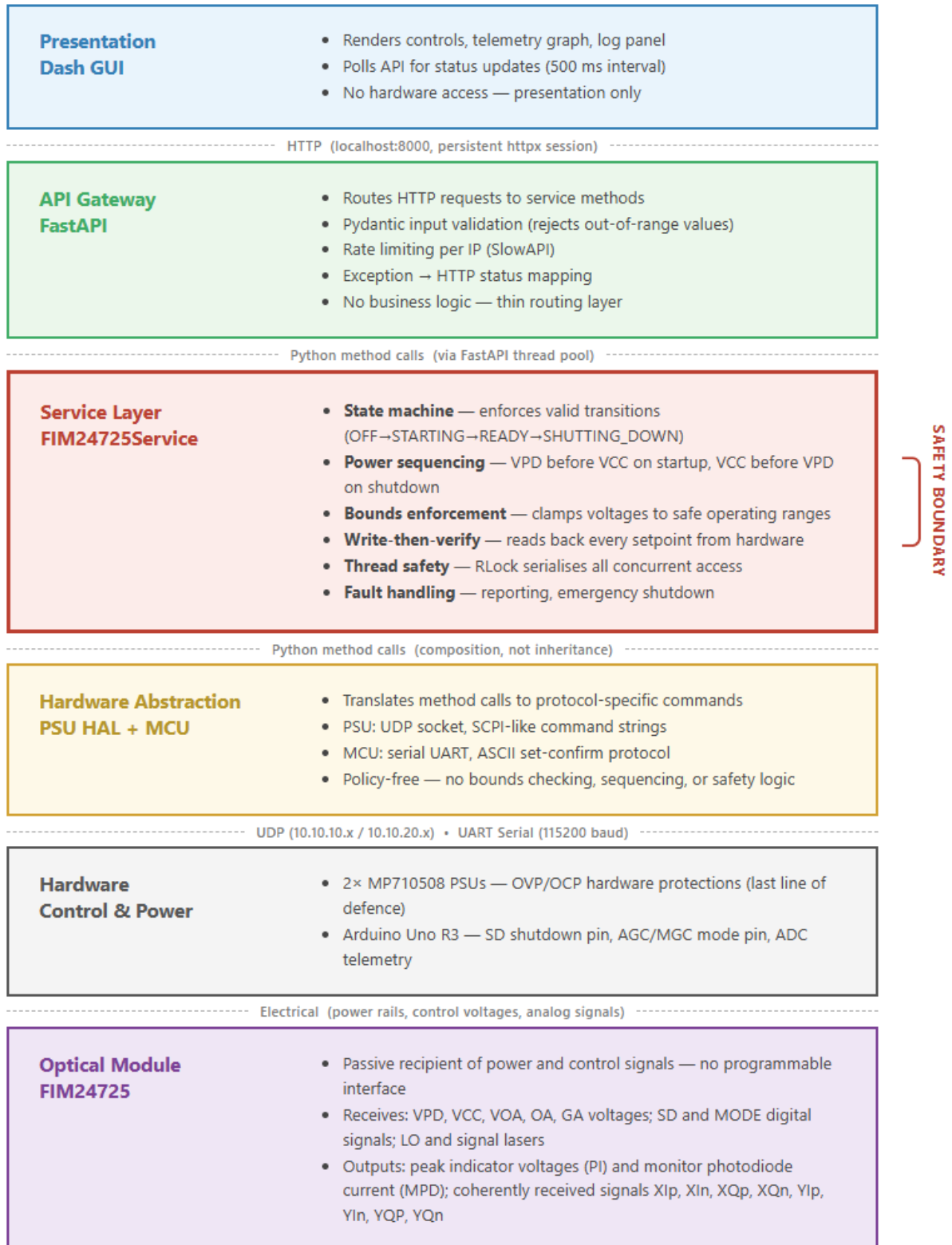


Figure 3.19: System Layer Architecture Diagram

Figure 3.19 represents the layered architecture of the control system. The critical property of this architecture is that the safety boundary is concentrated in a single layer. The service layer is the only

component that knows the startup sequence, the valid voltage ranges, the state machine transitions, and the fault recovery procedure. The layers above it (API and GUI) provide input validation as a convenience and usability measure, but could be removed entirely without compromising safety. The layer below it (HAL) is deliberately policy-free, executing whatever commands it receives without judgement, and the control hardware is the physical manifestation of these commands, providing the FIM24725 with power and setting analogue and digital controls.

Each layer has a single responsibility. This means any layer can be replaced, swapping the Dash GUI for a different frontend would require zero changes to the backend. The same software, with only environment variable changes, can run on a different host, communicate with different PSU models (given a new HAL driver), or use a different MCU.

Packaging it all together

To create a unified plug-and-play box, a custom PCB was designed to route ICR, MCU, and PSU connections. The PCB was designed as a compact interface board to keep I/O organized and accessible. It features 0.2 mm tracks which are suitable for the mostly low-current signal routing, and was also adequate for the maximum expected current of 400 mA (TIA VCC) over short distances. It also features mounts for the pull-down and voltage divider resistors required for the MCU readback and control signals and a large ground plane to reduce electrical noise. It was also equipped with mounting holes to facilitate mounting within a larger enclosure.

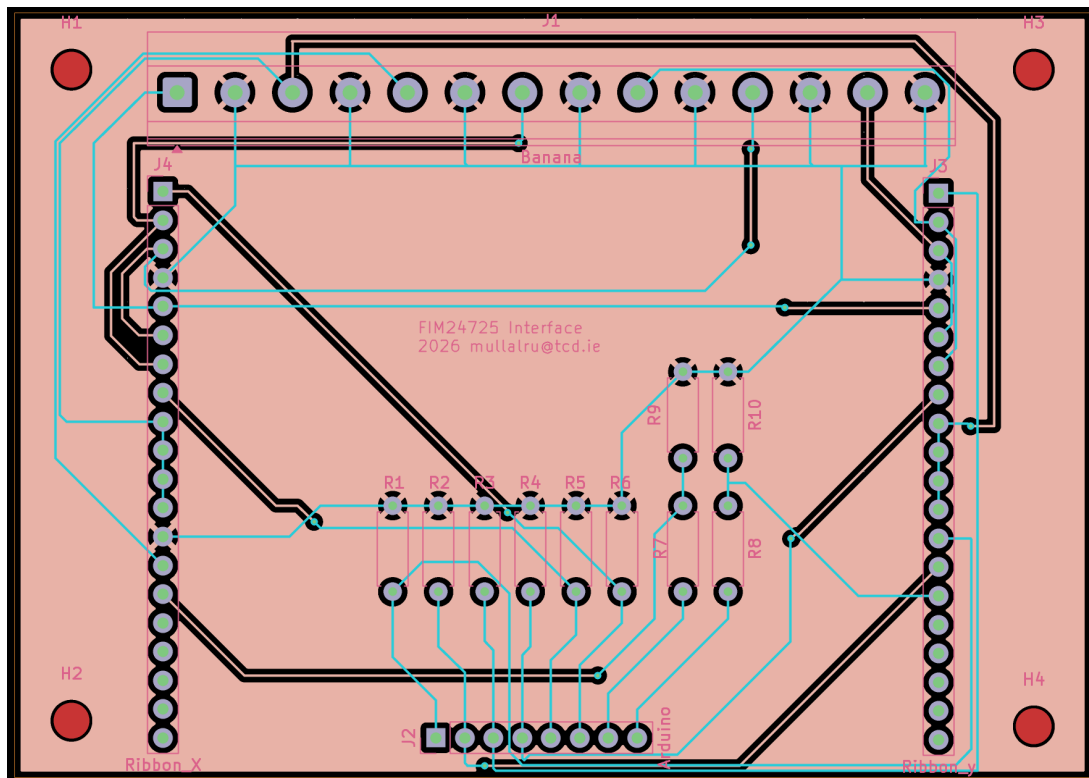


Figure 3.20: Custom Routing PCB Design Gerber.

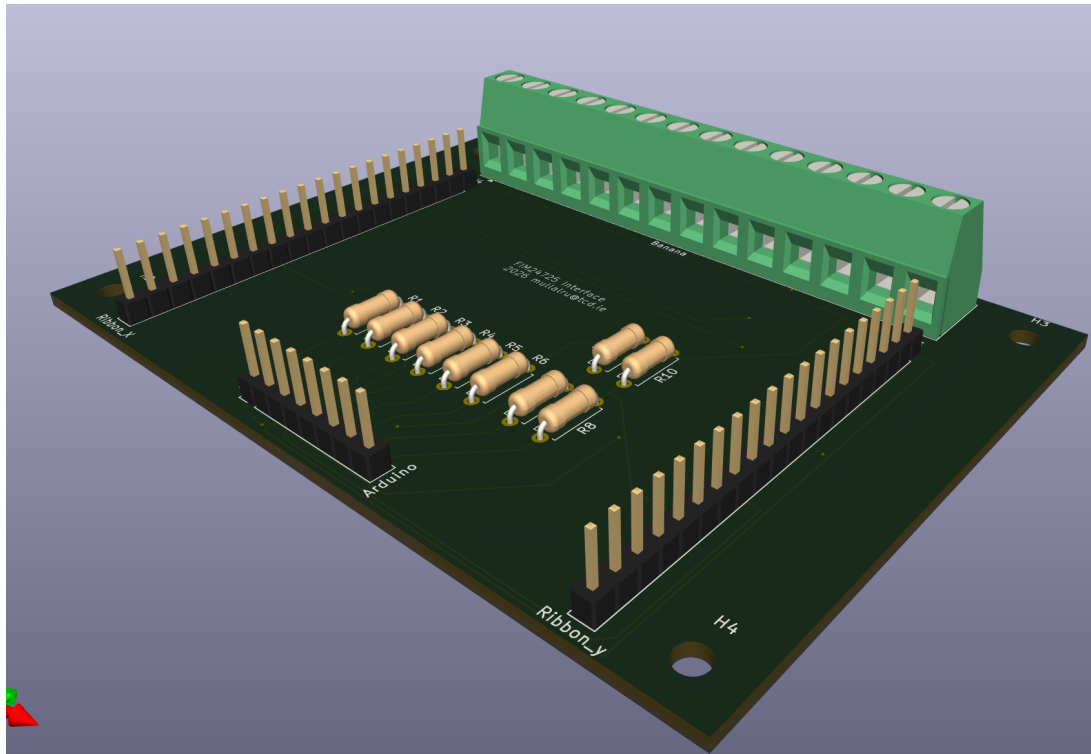


Figure 3.21: Custom Routing PCB 3D View.

The ICR, MCU, PCB and Raspberry Pi unifiedly placed inside an enclosure with banana connector female connectors mounted on the enclosure to connect the PSUs (allowing them to be easily disconnected and reconnected for other use). Input connectors for optical fibre, and GPPO to SMA output connectors were also mounted on the enclosure facilitating the connection of the laser inputs to the receiver and the electrical output signals to a DAC or other analysis equipment.



Figure 3.22: Assembled enclosure showing external connections.

The result is a self-contained unit that presents a clean external interface: banana sockets for PSU connections, LC fibre connectors for optical input, and SMA connectors (via GPPO-to-SMA adapters) for RF output. A researcher setting up the receiver connects to the PSU banana sockets and ethernet, fibres to the optical inputs, SMA cables to their measurement equipment. The entire physical setup takes approximately 10 minutes from unpacked components to a system ready for software startup compared to the previous workflow, which required identifying individual wires on the evaluation board's DC cables, tracing them to the correct PSU channels using the datasheet pinout, and manually verifying every connection before powering on. The receiver becomes a laboratory instrument that can be moved between benches, shipped to a partner institution, or stored and returned to service without re-learning its wiring. Note that Figure 3.22 is not the final system, but a similar one used for illustrative purposes.

4 Evaluation

This chapter evaluates the control system against the design requirements in Section 3.1, drawing on three sources of evidence: hardware integration testing performed with real PSUs and Arduino on a simulated physical load, operational use during a coherent detection experiment, testimonial from a user. A 'requirements traceability' table is given in Appendix A1.3.

4.1 Coherent Detection Experiment

To validate that the control system enables correct operation of the FIM24725 ICR, a coherent detection experiment was conducted. The objective was to confirm that the receiver, under the control system, produces the expected electrical beat frequency when a signal laser and local oscillator are mixed in the optical hybrid.

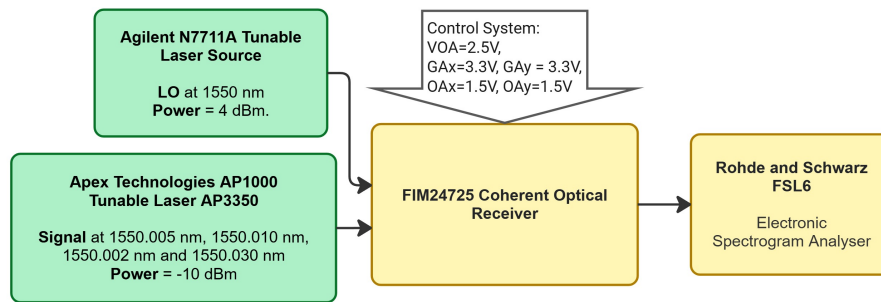


Figure 4.1: Coherent Detection Experimental Setup.

The LO wavelength was held fixed at 1550.000 nm. The signal laser was tuned to a series of wavelength offsets: 0.005, 0.010, 0.020, 0.030, and 0.040 nm. At each offset, the optical spectra of both lasers were taken (Figure 4.2), and the electronic spectrogram analyser was used to capture the electrical beat frequency at the receiver's XI and YI outputs (Figure 4.3).

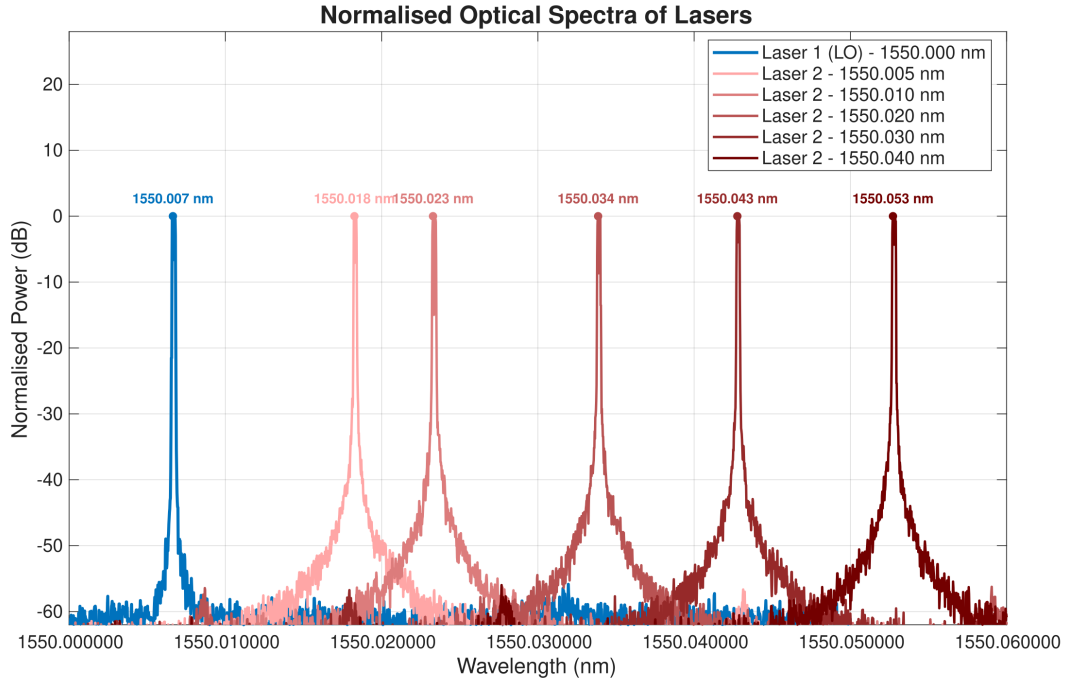


Figure 4.2: Normalised Optical Spectra of Lasers used in Experiment.

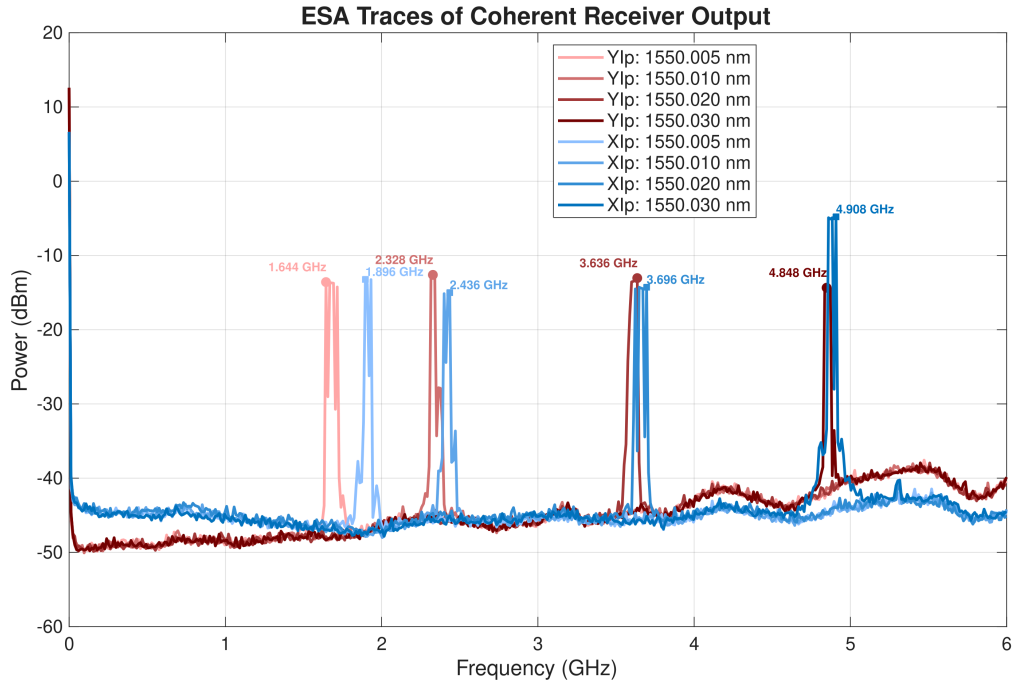


Figure 4.3: ESA Traces of Coherent Receiver Output from Experiment.

Table 4.1: Experimental measured and theoretical frequency differences.

Condition	Measured λ (nm)	$\Delta\lambda$ (nm)	Theoretical Δf (GHz)	Measured Δf Ylp (GHz)	Error (GHz)	Measured Δf Xlp (GHz)	Error (GHz)
LO 1550 nm	1550.01	0	0	N/A	N/A	N/A	N/A
Signal 1550.005 nm	1550.02	0.011617	1.451	1.644	0.193	1.896	0.445
Signal 1550.01 nm	1550.02	0.016706	2.086	2.328	0.242	2.436	0.350
Signal 1550.02 nm	1550.03	0.027334	3.413	3.636	0.223	3.696	0.283
Signal 1550.03 nm	1550.04	0.036164	4.516	4.848	0.332	4.908	0.392
Signal 1550.04 nm	1550.05	0.046224	5.772	N/A	N/A	N/A	N/A

Theoretical beat frequencies were calculated using Equation 2.3, and yielded the results in Table 4.1. Due to the tunable laser sources outputting higher values than specified as per the optical spectra, the 0.04 nm offset could not be measured on the ESA with a bandwidth of 6/GHz.

The experimental results demonstrate the relationship between wavelength offset ($\Delta\lambda$) and the resulting beat frequency (Δf) observed at the output of the coherent receiver. As wavelength separation increases, the beat frequency increases correspondingly. Measured values of beat frequency exceeded theoretical calculations by 0.193 - 0.445 GHz. This discrepancy is unlikely to arise from the wavelength-frequency approximation, whose error is negligible at these scales. It can likely be attributed to environmental factors such as temperature and laser cavity fluctuations causing laser drift, which was observable on the ESA. The presence of a well-defined and tunable beat frequency provides strong evidence of coherent detection.

4.2 User Testimonial

Beyond formal validation, the system was used operationally by a laboratory colleague during the coherent detection experiment. The colleague was added to the Tailscale tailnet and was able to access and operate the control system within minutes, with no prior experience of the software and no installation beyond the Tailscale client. After using the system across multiple experimental sessions, they provided the following feedback:

"In my experience, while the time saving compared with a fully manual setup is not especially large, the system is much more convenient to use. The power supplies are bulky and not very portable, so they often cannot be placed close to the experiment. Being able to control them from a laptop makes things much easier, particularly since the monitors are usually in a different location. It also saves time because you do not need to keep moving back and forth to the power supplies to change voltages or run current sweeps while checking the monitors. This is especially useful when trying to optimise values while observing the monitor output, since you can do both simultaneously from a small portable device. Overall, it makes the setup and control of the experiment much easier."

4.3 Integration Testing

The safety architecture is the core contribution of this project. Validation was performed by integrating the real PSUs and Arduino with a simulated resistive load in place of the FIM24725, and systematically attempting to induce every failure mode that could be anticipated. The following is not an exhaustive list.

PSU communication loss. A PSU Ethernet cable was disconnected during operation. The next command to that PSU timed out after the configured period, the service raised a `VerificationError`, the state machine transitioned to `FAULT`, and the could be cleared after a shutdown when the PSU was reconnected.

MCU disconnection. The Arduino USB cable was disconnected during `READY` state. The background reader thread detected the serial exception, the next telemetry read timed out, and the service transitioned to `FAULT` with an `MCUError`. Fault could be cleared with best effort shutdown.

Out-of-range voltage submission. Voltages outside the permitted ranges were submitted via the API (to bypass the browser's input constraints). A VOA request of 5.5 V was rejected with HTTP 422 by the Pydantic validation layer before reaching the service. Bypassing the API, a VOA request of 4.9 V was clamped to 4.8 V by the service layer's bounds validation, with a warning logged. In neither case did an unsafe voltage reach the hardware.

Operation in wrong state. A `POST /controls/voa` request was issued while the system was in `OFF` state. The service rejected it with a `StateError` (HTTP 409), confirming that the state machine prevents control operations outside of `READY`.

Power cycle recovery. The Pi was power-cycled by disconnecting and reconnecting its supply. After approximately 60 seconds, the GUI was accessible via Tailscale. The system was in `OFF` state, with no hardware energised. The operator initiated startup normally.

4.4 Comparison With Other Systems

One of the motivations for this project was that existing control frameworks such as EPICS carry significant infrastructure overhead, while commercial instrument control solutions (National Instruments LabVIEW) carry significant licensing and hardware integration costs.

The two MP710508 power supplies are excluded from the table as they are shared laboratory equipment. The total cost of the control system hardware is approximately €143, all of which was sourced from existing laboratory inventory or salvaged components. The software stack is entirely open source with no licensing fees. By contrast, a single LabVIEW Professional Development licence costs excess of €3,000, and a National Instruments CompactDAQ chassis with appropriate I/O modules for this application would add a further costs in hardware [28]. The cost of the control

Table 4.2: Control system bill of materials.

Component	Source	Cost	Notes
Raspberry Pi 3 Model B	Lab inventory	~€37	Available on hand
OSEPP Arduino Uno R3	Lab inventory	~€29	Available on hand
USB-to-Ethernet adapter	Lab inventory	~€5	Second PSU subnet
Custom PCB	Fabricated	€29.13	Board fabrication + components
Enclosure + connectors	Salvaged	~€20	Estimated replacement cost
SD card (32 GB)	Lab inventory	~€8	Pi boot media
USB cables, Ethernet cables	Lab inventory	~€15	Standard cables
Total hardware		~€143.133	
<i>Software and services</i>			
Docker, FastAPI, Dash, Python	Open source	€0	No licensing costs
Tailscale	Free tier	€0	Up to 100 devices, 3 users
Total		~€143.13	

system described in this report is an order of magnitude lower than a comparable commercial solution.

Table 4.3: Comparison of control system approaches for a single-instrument laboratory deployment.

Criterion	EPICS	LabVIEW	This project
Safety enforcement	Alarms + interlocks; heavy per-device config	Non-structural; depends on developer discipline	Structural: FSM, sequencing, validation, verify
Deployment	High: IOC + CA + device modules	Moderate: runtime + server setup	Low: docker compose up -d
Configuration	Extensive: record DB (DSL)	Moderate: VI development; vendor drivers	Minimal: env vars + code-defined safety
Remote access	Native (Channel Access); needs infra	Add-on (web/remote panels); licensing	Native; no install
Cost	Free; infra + hardware required	>€3,000 + NI hardware	~€143; open source
Development time	Weeks–months (custom device support)	Days–weeks	8 weeks (end-to-end)
Reusability	High: framework + reusable modules	Moderate: hardware-coupled VIs	Moderate: reusable HAL; specific service

The comparison in Table 4.3 is not intended to argue that the approach taken in this project is universally superior. EPICS would provide stronger built-in alarm and archiving infrastructure, but the configuration overhead for a seven-rail, single-instrument deployment is disproportionate, and no existing device support module covers the MP710508 PSU. LabVIEW would provide rapid UI development, but its safety enforcement would depend on implementation discipline rather than architectural guarantees, it would introduce a significant licensing cost, and containerised deployment would not be practical. Ad-hoc scripting would have been the fastest path to basic functionality, but would provide no state management, no sequencing enforcement, and no protection against operator error: precisely the deficiencies that motivated this project.

4.5 Limitations

No formal software testing. The system was validated through manual fault injection and operational use rather than automated unit or integration tests. Formal tests would inspire more confidence in the system, especially after code changes.

ADC resolution. The Arduino's 10-bit ADC with a 3.3 V reference provides approximately 3.2 mV resolution over the 0–2 V PI signal range. This is adequate for monitoring but may be insufficient for precision characterisation measurements.

No application-level authentication. Access control relies entirely on Tailscale's network-level authentication. Any device on the tailnet can access the full control interface without further credential checks, and there is no audit trail of which user performed an action.

Single-instrument scope. The system is designed for the FIM24725 specifically. While the PSU HAL and MCU interface are reusable, the service layer's sequencing logic, rail registry, and state machine are specific to this receiver. Adapting the system to a different instrument would require a new service layer.

Limited user testing. The remote access trial involved a single colleague. No formal usability study was conducted with a broader group of users. While the colleague was able to use the system quickly, the sample size is too small to draw general usability conclusions.

5 Conclusion

5.1 Summary of Achievements

This project set out to replace the manual, error-prone operation of a high-value optical coherent receiver with an automated control system and user interface that enforces safe operating procedures, improves repeatability, and enables remote access. That objective has been met.

The delivered system is a self-contained, networked control platform for the Fujitsu FIM24725 micro-integrated coherent receiver. Critical safety power-up and power-down sequences that previously relied on operator knowledge and written procedures are architectural guarantees. It validates every voltage setpoint against the manufacturer's specifications before it reaches the hardware, verifies every hardware command through readback, and transitions to a fault state, supplying fault information, which can be cleared by an operator. The system is accessible from any device on the laboratory's Tailscale network through a standard web browser, with no client installation required.

The system occupies the middleground between a heavyweight facility-wide infrastructure like EPICS, and ad-hoc scripting approaches common in small laboratories. It achieves structural safety enforcement comparable to EPICS at a fraction of the deployment complexity, which would require IOCs, custom C device support, an SNL state machine, a 50+ record database, and learning multiple domain-specific tools, all for seven rails and two digital signals. It does it at a fraction of the cost of licensing LabVIEW (>€3,000) and purchasing compatible National Instruments hardware, on a platform that would require programming in a graphical language ill-suited to socket-level operations, and the LabVIEW runtime on every client machine practically precluding it from platform agnostic deployment, and browser-based access.

The total hardware cost of approximately €143 in commodity components, combined with an entirely open-source software stack, makes the approach accessible to any laboratory without significant investment. The system was designed, implemented, and deployed within eight weeks by a single developer.

The control system was used to conduct a coherent detection experiment in which the receiver successfully produced well-defined electrical beat frequencies across multiple wavelength offsets, confirming correct stable operation over multiple sessions. Differences in theoretical and measured beat frequencies were observed and were likely due to laser cavity fluctuations and environmental factors.

A laboratory colleague was added to the Tailscale network and was operating the system independently within minutes, with no prior experience of the software. Their feedback highlighted that the primary benefit was not time saving necessarily but the convenience of co-locating control and observation, rather than moving between the power supplies and measurement equipment.

The delivered system, comprising of a Raspberry Pi, Arduino, custom PCB, and the receiver housed

in a single enclosure transforms what was previously a loose collection of bench equipment into a portable, plug-and-play laboratory instrument.

5.2 Future Work

Replacing the Arduino Uno with an Ethernet-capable microcontroller (or adding a W5500 Ethernet shield) would eliminate the USB serial link, allowing the MCU to communicate over IP like the PSUs. This would simplify the topology of the deployment and decouple the MCU from the host's physical USB ports enabling deployment on a remote server rather than a co-located Pi if desired.

The Arduino's 10-bit ADC provides approximately 3.2 mV resolution over the peak indicator signal range and could enable characterisation measurements that could be used identify optimal gain settings from GA sweep data, by locating the gain value that maximises output amplitude without driving the output into clipping. Replacing it with a higher resolution ADC could enable measuring small performance differences between quadrature channels or tracking gradual degradation over time.

The PSU HAL and MCU interface are instrument-agnostic by design. Adapting the system to control a different receiver or optical device would require a new service layer but could reuse the HAL, API framework, GUI structure, and deployment infrastructure.

5.2.1 Wider Societal Implications

This project does not raise direct ethical concerns. It processes no personal data, poses no environment or dual-use risk, and operates in a controlled environment. However, the work this project facilitates has broad societal implications. Coherent detection is one of the enabling technologies behind modern high-capacity fibre-optic networks, which carry the majority of the world's internet, telecommunications, and data centre traffic. While the project itself does not advance coherent transmission research, it provides laboratory infrastructure that enables research to be conducted more reliably, more safely, and by a wider group of researchers. Lowering the barrier to experimental work, even incrementally, contributes to the broader effort of continued growth in network capacity on which modern society increasingly depends.

Bibliography

- [1] Fujitsu Optical Components Limited. Micro Integrated Coherent Receiver Application Note. Kawasaki, Japan: Fujitsu Optical Components Limited; 2021. PS-15-349-05. Confidential document. Applies to FIM24721, FIM24731, FIM24723, FIM24733, FIM24725, and FIM24735 series.
- [2] Farnell. Bench Power Supply, Programmable, 4 Channel, 5A; 2026. Accessed: 2026-04-02. Available from: <https://uk.farnell.com/multicompro/mp710508/bench-power-supply-prog-4-ch-5a/dp/3514779>.
- [3] finally opening my arduino compatible osepp uno r3 plus 2019; 2019. Available from: <https://newscrewdriver.com/2019/04/29/finally-opening-my-arduino-compatible-osepp-uno-r3-plus/>.
- [4] Raspberry Pi Foundation. Raspberry Pi 3 Model B; 2026. Accessed: 2026-04-02. Available from: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>.
- [5] Agrawal GP. Fiber-Optic Communication Systems. Wiley; 2021.
- [6] Kikuchi K. Fundamentals of Coherent Optical Fiber Communications. Journal of Lightwave Technology. 2016 Jan;34(1):157–179. Available from: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7174950>.
- [7] Ip EM, Kahn JM. Fiber Impairment Compensation Using Coherent Detection and Digital Signal Processing. Journal of Lightwave Technology. 2010 Feb;28(4):502–519. Available from: <https://opg.optica.org/jlt/abstract.cfm?uri=jlt-28-4-502>.
- [8] Fujitsu Optical Components Limited. FIM24725 Micro Integrated Coherent Receiver (Rev. A). Fujitsu Optical Components Limited; 2019. FIM24725 Rev. A.
- [9] standard commands for programmable instruments (scpi) volume 1: syntax and style 1999; 1999. Available from: <https://previous.ivifoundation.org/docs/scpi-99.pdf>.
- [10] Dalesio LR, Hill JO, Kraimer M, Lewis S, Murray D, Hunt S, et al. The experimental physics and industrial control system architecture: past, present, and future. Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment. 1994 Dec;352(1-2):179–184.
- [11] Lange R. An arena for cooperation; 2019. Available from: <https://www.iter.org/node/20687/arena-cooperation>.
- [12] ITER. CODAC; 2023. Available from: <https://www.iter.org/machine/supporting-systems/codac>.
- [13] EPICS. about epics - epics controls 2024; 2024. Epics-controls.org. Available from: <https://epics-controls.org/about-epics>.

- [14] ELLIOTT C, VIJAYAKUMAR V, ZINK W, HANSEN R. National Instruments LabVIEW: A Programming Environment for Laboratory Automation and Measurement. Journal of the Association for Laboratory Automation. 2007 Feb;12(1):17–24. Available from: <https://journals.sagepub.com/doi/full/10.1016/j.jala.2006.07.012>.
- [15] NI. How to Choose the Right NI Software License; 2026. Available from: <https://www.ni.com/en/shop/software-portfolio/choose-right-software-license.html>.
- [16] Grecco HE, Dartailh MC, Thalhammer-Thurner G, Bronger T, Bauer F. PyVISA: the Python instrumentation package. Journal of open source software. 2023 Apr;8(84):5304–5304.
- [17] Casagrande S. Welcome to InstrumentKit Library’s documentation! — InstrumentKit Library 1.0.0b3.dev26+gc976204e0 documentation; 2025. Available from: <https://instrumentkit.readthedocs.io/en/latest/>.
- [18] MathWorks. What Is a State Machine?. MathWorks; 2025. Mathworks.com. Available from: <https://uk.mathworks.com/discovery/state-machine.html>.
- [19] Reidt T. What is the Hardware Abstraction Layer (HAL)?; 2022. emteria.com. Available from: <https://emteria.com/learn/hardware-abstraction-layer>.
- [20] GeeksforGeeks. Separation of Concerns (SoC); 2024. GeeksforGeeks. Available from: <https://www.geeksforgeeks.org/software-engineering/separation-of-concerns-soc/>.
- [21] IBM. REST APIs; 2021. Ibm.com. Available from: <https://www.ibm.com/think/topics/rest-apis>.
- [22] AWS. What is Containerization? Containerization Explained - AWS; 2025. Amazon Web Services, Inc. Available from: <https://aws.amazon.com/what-is/containerization/>.
- [23] Ramírez S. FastAPI Documentation; 2024. Available from: <https://fastapi.tiangolo.com/>.
- [24] Matherly J. Shodan: The Search Engine for Internet-Connected Devices; 2024. Available from: <https://www.shodan.io/>.
- [25] Tailscale Inc . Tailscale Documentation; 2024. Available from: <https://tailscale.com/docs>.
- [26] Plotly Technologies Inc . Dash Documentation & User Guide; 2024. Available from: <https://dash.plotly.com/>.
- [27] Docker Inc . Python — Official Docker Image; 2024. Available from: https://hub.docker.com/_/python.
- [28] National Instruments. Select Your NI LabVIEW Edition; 2024. Available from: <https://www.ni.com/en/shop/labview.html>.

A1 Appendix

A1.1 MP710508 SCPI Schema

The first page of the available SCPI commands are included here for reference.

1.LOCK:<Boolean>

Description: locking the buttons.

Example:LOCK:1

Locking the buttons.

2.ISET<X>:<NR2>

Description: setting the current value.

Example:ISET1:2.5

Setting the current on CH1 as 2.5A.

3.ISET<X>?

Description: querying the setting value of the channel current.

Example:ISET1 ?

Returning to the CH1 current setting value.

4.VSET<X>:<NR2>

Description: setting the voltage value.

Example:VSET1:12.5

Setting CH1 as 12.5V.

5.VSET<X>?

Description: querying the channel voltage setting value.

Example:VSET1 ?

Returning to the CH1 voltage setting value.

6.IOUT<X>?

Description: querying the channel current value

Example:IOUT1 ?

Returning to CH1 current value

A1.2 Fujitsu Micro Integrated Coherent Receiver Application Note: Pin functions and power sequencing

Pin functions of Micro ICR



DC pins

No.	NAME	FUNCTION
1	RFU	Reserved for Future Use
2	RFU	Reserved for Future Use
3	MC/AGC	MC/AGC Selection
4	MPD+	Monitor Photodiode Cathode
5	MPD-	Monitor Photodiode Anode
6	VPD_YIn	Photodiode power supply (YIn)
7	VPD_YIp	Photodiode power supply (YIp)
8	VPD_YQp	Photodiode power supply (YQp)
9	VPD_YQn	Photodiode power supply (YQn)
10	PI-YI	Peak Indicator YI
11	GA-YI	Gain Adjust YI
12	OA-YI	Output Amplitude Adjust YI
13	VCC_Y	Amplifier power supply (YI and YQ)
14	GND	Ground
15	OA-YQ	Output Amplitude Adjust YQ
16	GA-YQ	Gain Adjust YQ
17	PI-YQ	Peak Indicator YQ
18	PI-XI	Peak Indicator XI
19	GA-XI	Gain Adjust XI
20	OA-XI	Output Amplitude Adjust XI
21	GND	Ground
22	VCC_X	Amplifier power supply (XI and XQ)
23	OA-XQ	Output Amplitude Adjust XQ
24	GA_XQ	Gain Adjust XQ
25	PI-XQ	Peak Indicator XQ
26	VPD_XIn	Photodiode power supply (XIn)
27	VPD_XIp	Photodiode power supply (XIp)
28	VPD_XQp	Photodiode power supply (XQp)
29	VPD_XQn	Photodiode power supply (XQn)
30	VOA2	VOA GND
31	VOA1	VOA control
32	SD	Shutdown
33	RFU	Reserved for Future Use
34	RFU	Reserved for Future Use

: Control pins
 : Monitor pins
 : Power supply pins

RF pins

No.	Name
1	GND
2	XIp
3	XIn
4	GND
5	XQp
6	XQn
7	GND
8	YIp
9	YIn
10	GND
11	YQp
12	YQn
13	GND

Recommended power up sequence of Micro ICR



The Micro ICR should be powered up with the following sequence.

1. Prevent any electrical surge with earth band etc.
2. Confirm the Micro ICR and evaluation board are not connected to any evaluation system. i.e. DC/ AC power supplies/ monitors, optical outputs and other equipment
3. Turn on main switches of DC/ AC power supplies/ monitors.
4. Set DC power supplies to 0 [V]
5. Operate DC power supplies outputs
6. Connect the Micro ICR and evaluation board to evaluation system. i.e. DC/ AC power supply/ monitor and optical input
7. Set photodiode power supplies;
VPD_XIp, VPD_XIn, VPD_XQp, VPD_XQn,
VPD_YIp, VPD_YIn, VPD_YQp, VPD_YQn,
from 0 [V] to +5 [V] gradually.
8. Set amplifier power supplies ;
VCC_X, VCC_Y,
from 0 [V] to +3.3 [V] gradually.
9. Turn on optical input.
10. Adjust control terminal voltages;
GA_XI, GA_XQ, GA_YI, GA_YQ,
OA_XI, OA_XQ, OA_YI, OA_YQ,
MC/AGC, SD,
to get proper performance.

Note: Step 8 can be done simultaneously with step 7 though, if amplifier power supplies are up prior to photodiode power supplies, the amplifiers and photodiodes will be seriously damaged.

Recommended power down sequence of Micro ICR

The Micro ICR should be powered down with following sequence.

1. Turn off optical input
2. Set control terminal voltages to 0[V] gradually.
3. Set amplifier power supplies to 0[V] gradually.
4. Set photodiode power supplies to 0[V] gradually.
5. Disconnect receiver and evaluation board from evaluation system. i.e. DC/AC power supplies/monitors and optical input.
6. Stop to operate DC/AC power supplies/monitors.
7. Turn off main switches of DC/AC power supplies/monitors.

Note: Step 3 can be done simultaneously with step 2 though, If photodiode power supplies are down prior to amplifier power supplies, the photodiodes and amplifiers will be seriously damaged.

A1.3 Requirements Traceability

- **Test** — the requirement was deliberately exercised by injecting specific conditions (e.g. disconnecting a PSU to trigger fault handling).
- **Demonstration** — the requirement was observed to be satisfied during normal operational use (e.g. successful startup and shutdown across multiple sessions).
- **Inspection** — the requirement is satisfied structurally by the design and can be verified by examining the code or configuration (e.g. hard-coded sequencing order).

Req.	How satisfied	Verified
FR1.1	Startup/shutdown enforce ordering (Algorithms 1, 2)	Demo
FR1.2	Service starts in OFF; POST required to enable	Inspect
FR1.3	OVP/OCP configured and verified before enabling output	Demo
FR1.4	PSU front panels locked during operation	Demo
FR1.5	Five-state FSM gates all operations	Test
FR1.6	Fault info with timestamp shown in GUI snapshot	Test
FR1.7	Fault state allows best-effort emergency shutdown	Test
FR2.1	Runtime voltage control via API endpoints and GUI controls	Demo
FR2.2	Pydantic rejects invalid input; service clamps as fallback	Test
FR2.3	Mode switching with echo verification	Demo
FR3.1	500 ms Arduino telemetry with client-side graph rendering	Demo
FR3.2	Structured logging with rotation and GUI access	Demo
FR4.1	Web-based single-page interface	Demo
FR4.2	State-based control enable/disable with confirmation modals	Demo
NFR1.1	MCU/PSU errors escalate to FAULT within one command cycle	Test
NFR1.2	Write-then-verify after every hardware command	Demo
NFR1.3	RLock serialises all concurrent access	Inspect
NFR1.4	Lock serialisation ensures command ordering; background telemetry thread decoupled from command path	Inspect
NFR2.1	Four-layer architecture with safety in service layer	Inspect
NFR2.2	Policy-free HAL reusable across instruments	Inspect
NFR2.3	Docker restart policy with systemd	Test
NFR2.4	Browser-based GUI requiring no client installation	Demo
NFR2.5	Same Docker images used on Pi and development machine	Demo
NFR2.6	All deployment parameters configurable via environment variables without code changes	Demo
NFR2.7	State-driven controls and checks with modals	Demo
NFR3.1	Tailscale ACL restricts access to authenticated members	Test
NFR3.2	API on localhost; GUI via Tailscale; no public exposure	Inspect

A1.4 Use of GenAI in this work.

I acknowledge the use of Claude Opus 4.6 (Anthropic, <https://claude.ai>), and ChatGPT 5.3 (OpenAI, <https://chatgpt.com/>). These tools were used in a supportive capacity.

Claude Opus and ChatGPT were used to assist with:

- debugging and syntax during coding tasks,
- structures for organising sections of the report,
- rephrasing sentences to improve clarity and readability.

All technical content, analysis, results, interpretations, and final wording were produced by the author.

Example Prompt:

Turn this into an overleaf table i can insert:

Measured λ (nm) $\Delta\lambda$ (nm) Theoretical Δf (GHz) Measured Δf YIp (GHz) Error in Δf (GHz)
Measured Δf XIp (GHz) Error in Δf (GHz) LO at 1550 nm 1,550.01 0 0 N/A N/A N/A N/
A Signal at 1550.005 nm 1,550.02 0.011617 1.451 1.644 0.193 1.896 0.445 Signal at
1550.01 nm 1,550.02 0.016706 2.086 2.328 0.242 2.436 0.35 Signal at 1550.02 nm
1,550.03 0.027334 3.413 3.636 0.223 3.696 0.283 Signal at 1550.03 nm 1,550.04
0.036164 4.516 4.848 0.332 4.908 0.392 Signal at 1550.04 nm 1,550.05 0.046224
5.772 N/A N/A N/A N/A

Example Response:

Here is a clean LaTeX table version using tabularx so it fits nicely:

```
“‘‘latex
\usepackage{tabularx} % in preamble

\begin{table}[htbp]
\centering
\caption{Measured and theoretical frequency differences.}
\label{tab:freq-results}
\small
\renewcommand{\arraystretch}{1.15}

\begin{tabularx}{\linewidth}{@{}l c c c c c c c@{}}
\hline
\textbf{Condition} & \textbf{Measured  $\lambda$  (nm)} & \textbf{ $\Delta\lambda$  (nm)} & \textbf{Theoretical  $\Delta f$  (GHz)} & \textbf{Measured  $\Delta f$  YIp (GHz)} & \textbf{Error (GHz)} & \textbf{Measured  $\Delta f$  XIp (GHz)} & \textbf{Error (GHz)} \\
\hline
LO at 1550 nm & 1550.01 & 0 & 0 & N/A & N/A & N/A & N/A \\
Signal at 1550.005 nm & 1550.02 & 0.011617 & 1.451 & 1.644 & 0.193 & 1.896 & 0.445 \\
Signal at 1550.01 nm & 1550.02 & 0.016706 & 2.086 & 2.328 & 0.242 & 2.436 & 0.350 \\
Signal at 1550.02 nm & 1550.03 & 0.027334 & 3.413 & 3.636 & 0.223 & 3.696 & 0.283 \\
Signal at 1550.03 nm & 1550.04 & 0.036164 & 4.516 & 4.848 & 0.332 & 4.908 & 0.392 \\
Signal at 1550.04 nm & 1550.05 & 0.046224 & 5.772 & N/A & N/A & N/A & N/A
\end{tabularx}
```

```

Signal at 1550.03 nm & 1550.04 & 0.036164 & 4.516 & 4.848 & 0.332 & 4.908 & 0.392 \\
Signal at 1550.04 nm & 1550.05 & 0.046224 & 5.772 & N/A & N/A & N/A & N/A \\
\hline
\end{tabularx}
\end{table}
'''

---

### Small improvements I made

* Removed commas in numbers ('1,550.01 →1550.01') →avoids LaTeX spacing issues
* Shortened repeated headers ('Error in Îf →Error')
* Used math mode for symbols ('$λ$, $Δ f$')
* Aligned everything to fit neatly within 'linewidth'

---

If you want, I can also:

* Align numbers by decimal point ('siunitx' - looks *much* cleaner)
* Split YIp/XIp into subcolumns with a multicolumn header (more professional report
  style)

```